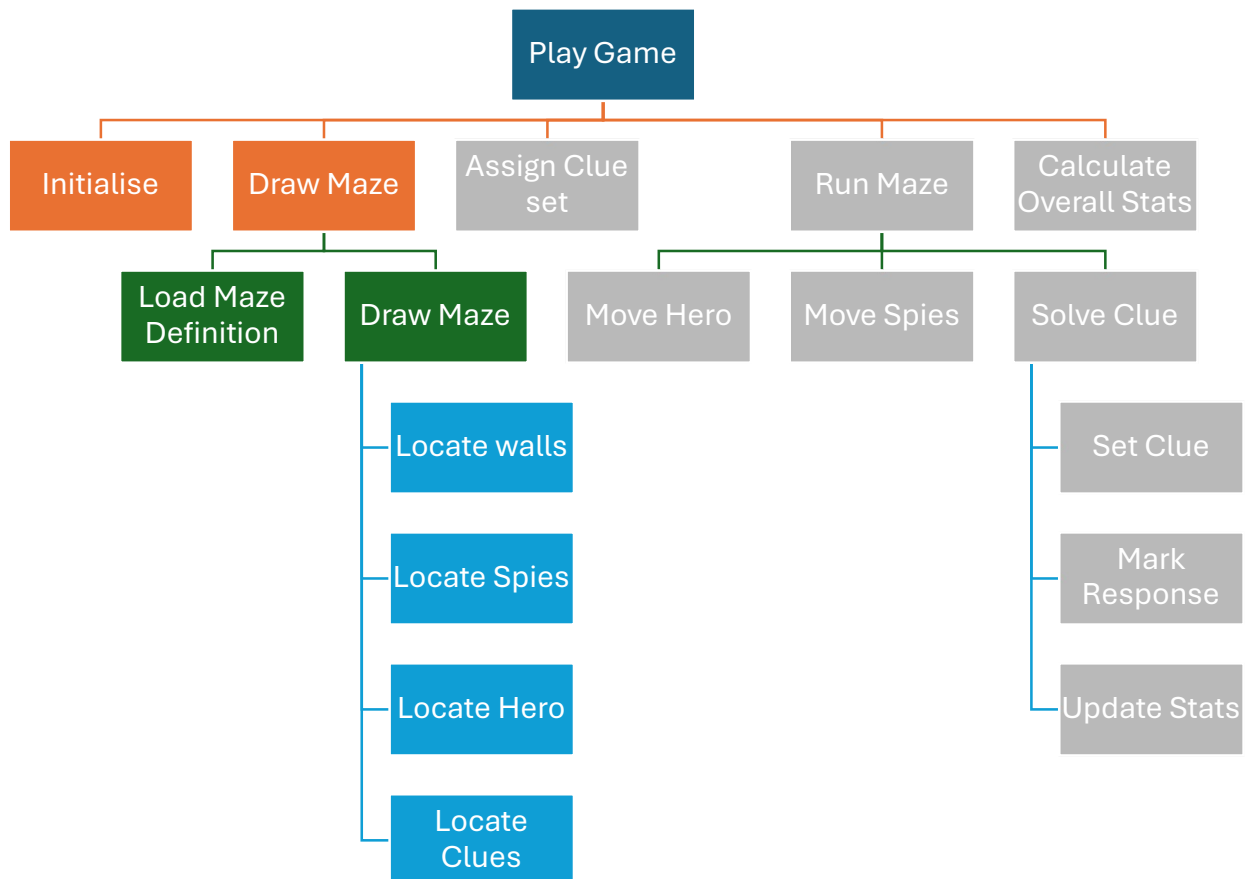


36. 2nd Iteration: Game Operation

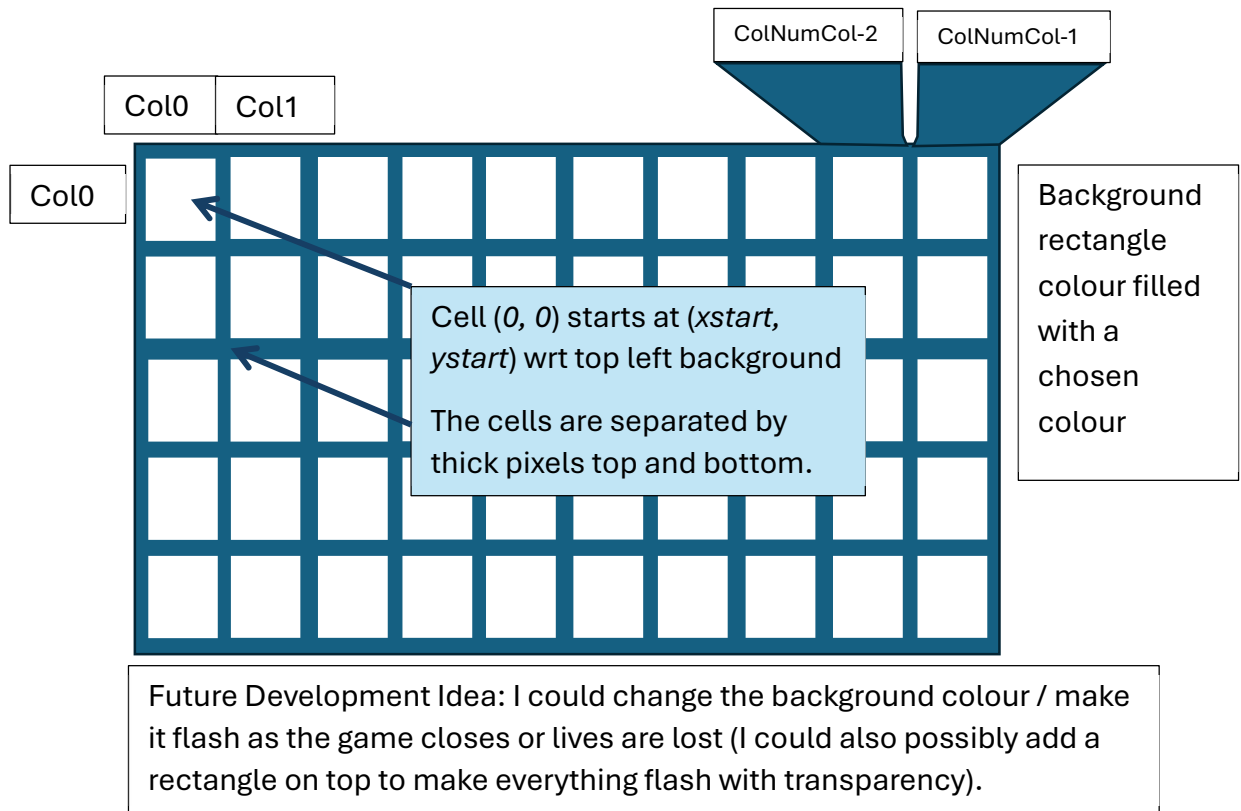
36.1. I am now going to continue considering the different elements of my decomposed system to begin to implement my game screen and the initialise function. I will be focussing on the areas within the coloured boxes on the diagram below:



36.2. This is how I would like the sub programmes in my game to operate:
Given the MazeName and UserName as input parameters, the algorithm is:

- Initialise Game:
 - o Set default stats: Score (0), Lives (3)/Health (100%), Time Elapsed (00:00)
 - o Set cell sizes: cell width, cell height (pixels)
 - o Load maze file:
 - Query the database with the MazeName
 - Query the database for the associated ClueSe
 - Set NumRows and NumCols from the maze definition to draw it
 - o Load the Avatar
 - Query the database with the Username
- Draw Maze (repeated each update to create the animation affect)
 - o Show stats
 - o Draw the grid NumRows x NumCols
 - o Fill cells:
 - Fill Wall cells with wall colour
 - Fill Hero cells with Hero x colour/avatar graphic
 - Fill Spy cells with Spy colour
 - Fill Clue cells with clue colour / Clue graphic (different levels of difficulty)
 - Fill Empty Cells with Empty Colour
- Manage a Hero Move
 - o Change position for up/down/left/right arrow event
 - o Respond to move:
 - If collided with clue:
 - Pause TimeTaken
 - Display Clue
 - Get Answer
 - Check Answer
 - o increase score if correct
 - o (decrease score if incorrect)
 - o (Update Accuracy)
 - Remove clue so it cannot be asked again
 - If collided with an spy:
 - Decrement lives/Health
 - Check for game over
 - o Move a spy (repeated for all spies)
 - Change position
 - Asses impact of move:
 - If collided with hero
 - o Decrement lives/health
 - o Check for game over
- Refresh display – draw maze (reuse subprogram)
- I will develop the methods to draw the maze and the entities in it from a series of prototypes

36.3. **Drawing the Grid:** I will use the pygame module to draw a grid for the basis of the maze with borderless rectangles. To do this, I drew on the information in this reference <https://stackoverflow.com/questions/33963361/how-to-make-a-grid-in-pygame>¹⁶ which provides learning on drawing a grid in pygame.



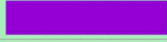
36.4. To define the cell (row number, column number):

- The top left corner of cell (0, 0) can be at (thick, thick)
- The top left corner of cell (0, 1) is at ((thick+width+thick), thick) = ((2*thick+width), (thick))
- The top left corner of cell (0, NumCols-1) (End of first row) is at ((NumCols)*thick + (NumCols-1)*width), thick)
- The top left corner of cell (NumRows-1, NumCols-1) (last cell) is at ((NumCols)*thick + (NumCols-1)*width, (NumRows)*thick + (NumRows-1)*width))
- The dimensions of the background are:
 - Width = (NumCols * width) + ((NumCols+1) * thick)
 - Height = (NumRows * width) + ((NumRows+1) * thick)

36.5. I used this algorithm as the basis of my grid in python:

```
If a square has dimensions width then  
To draw the grid:  
For Row = 1 to NumRows  
    For Col = 1 to NumCols  
        X = (Col)*thick + (Col-1)*width  
        Y = (Row)*thick + (Row-1)*width  
        Drawn box (width = width, height = width) at (X, Y)  
    Next Col  
Next Row
```

36.6. Pygame has a long list of named colours. This is a small collection of the colours within Pygame.color:

darkslategray4	
darkslategrey	
darkturquoise	
darkviolet	
deeppink	
deeppink1	
deeppink2	
deeppink3	
deeppink4	
deepskyblue	
deepskyblue1	
deepskyblue2	
deepskyblue3	
deepskyblue4	
dimgray	
dimgrey	
dodgerblue	

https://www.pygame.org/docs/ref/color_list.html¹⁷

Pygame Has a lot of named colours!

36.7. I created the maze graph using Pygame in Python as follows:

```
#TestGrid.py
#prototype to draw a grid in pygame
#Rosa 31/08/2024

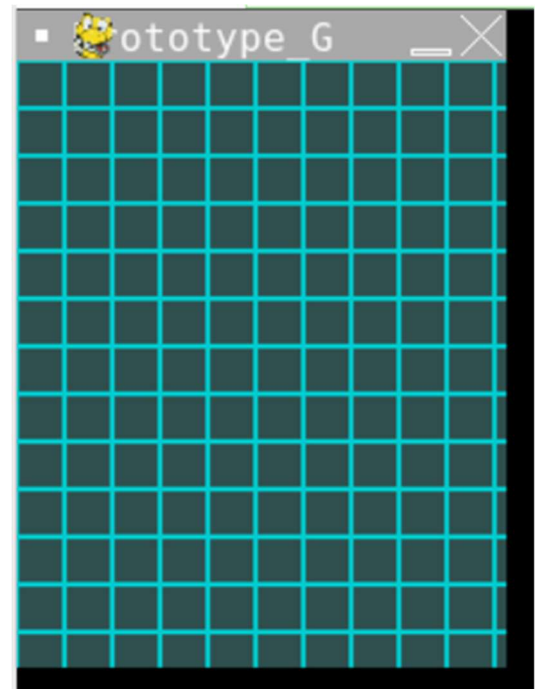
import pygame

#to be determined from maze definition
NumRows = 10
NumCols = 8

#for a cell
Width = 20 #pixels (square cell)
Thick = 5 #pixels (gap between cells)
CellColour = pygame.Color("darkturquoise")
#for the background
BackWidth = (NumCols * Width) + ((NumCols+1) * Thick)
BackHeight = (NumRows * Width) + ((NumRows+1) * Thick)
BackColour = pygame.Color("darkslategrey")
#Ref 17: https://www.pygame.org/docs/ref/color_list.html

#Set up a pygame window
pygame.init()
pygame.display.set_caption("Prototype_Grid1")
#iterate to draw grid
SCREEN = pygame.display.set_mode((BackWidth, BackHeight))
SCREEN.fill(BackColour)

for x in range(0, BackWidth, Width):
    for y in range(0, BackHeight, Width):
        rect = pygame.Rect(x, y, Width, Width)
        pygame.draw.rect(SCREEN, CellColour, rect, 1)
```



It works but the bottom row and right hand column don't finish cleanly – they look cut off.

I specified 10 rows and 8 columns. However, the output produced has 10 columns and 13 rows!

So there is an issue with drawing the cells.

```

for Row in range(1, NumRows+1):
    for Col in range(1, NumCols+1):
        X = (Col)*Thick + (Col-1)*Width
        Y = (Row)*Thick + (Row-1)*Width
        rect = pygame.Rect(X, Y, Width, Width)
        pygame.draw.rect(SCREEN, CellColour,
rect, 1)

```

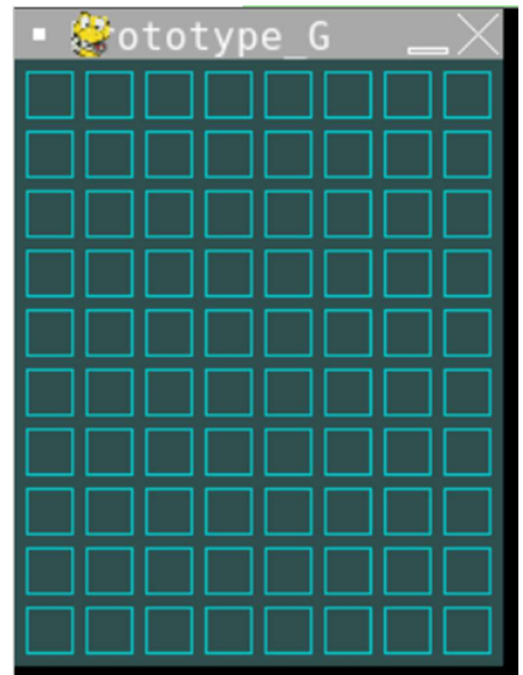
By making these changes, the Grid is now drawn correctly, but the cells are transparent.

```

pygame.draw.rect(SCREEN, CellColour,
rect, 0)

```

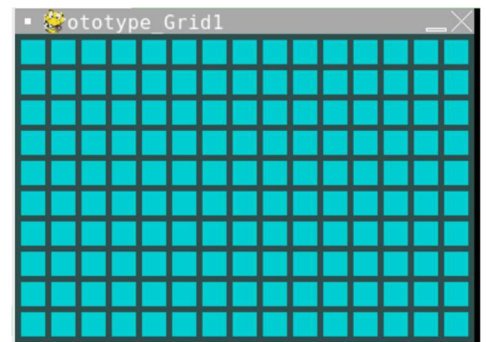
And changing the number 1 to 0 makes the cells a block colours



```

#define a maze
aMaze = [
["W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W"],
["W", "E", "C", "E", "E", "E", "E", "E", "S", "E", "C", "W", "C", "W"],
["W", "E", "W", "W", "W", "W", "W", "E", "W", "W", "W", "E", "W", "E", "W"],
["W", "E", "W", "E", "E", "S", "C", "E", "E", "W", "W", "E", "W", "E", "W"],
["W", "E", "W", "E", "W", "E", "W", "W", "E", "C", "W", "E", "E", "E", "W"],
["W", "E", "E", "E", "W", "E", "W", "E", "E", "W", "W", "E", "W", "E", "W"],
["W", "W", "W", "E", "W", "E", "W", "E", "E", "E", "E", "E", "W", "H", "W"],
["W", "E", "C", "E", "W", "E", "W", "E", "E", "W", "W", "W", "W", "E", "W"],
["W", "E", "S", "W", "E", "E", "C", "E", "E", "E", "E", "E", "C", "E", "W"],
["W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W"]
]

```



```

NumRows = len(aMaze)
#split the string in a row into a list of chars
aRow = aMaze[0][:] #get a string
aRow = str(aRow)
print(aRow)
aRow = aRow.replace(" ", "")
aRow = aRow.replace("'", "")
aRow = aRow.replace('"', "") #replace with null
print(aRow)
NumCols = len(aRow)

```

In order to get the correct number of Rows I needed to:

- get a row from the list
- cast the row to a string
- replace unwanted chars with ""
- count the number of chars in the string for NumCols.

The Correct grid is then drawn!

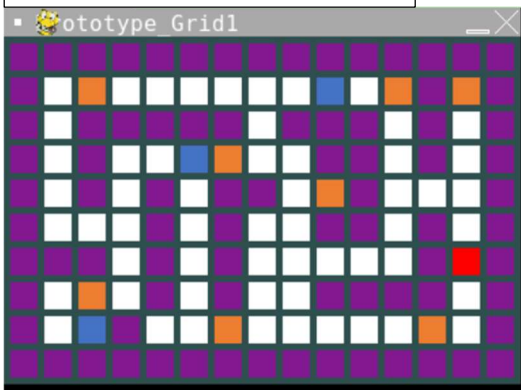
```
#define colours for different elements
colEmpty = "#FFFFFF"
colHero = "#FF0000"
colSpy = "#4472C4"
colClue = "#ED7D31"
colWall = "#821890"
```

A selection is used to define the colour of each cell.

```
#need to select cell colour
theChar = aRow[Col-1]
#selection
if theChar == "W":
    CellColour = colWall
elif theChar == "H":
    CellColour = colHero
elif theChar == "S":
    CellColour = colSpy
elif theChar == "C":
    CellColour = colClue
else:
    CellColour = colEmpty
```

I have added the colour definitions for the development maze.

My Prototype vs Maze



Both Mazes match!

W	W	W	W	W	W	W	W	W	W	W	W	W	W	W
W	E	C	E	E	E	E	E	S	E	C	W	C	W	W
W	E	W	W	W	W	W	E	W	W	W	E	W	E	W
W	E	W	E	E	S	C	E	E	W	W	E	W	E	W
W	E	W	E	W	E	W	W	E	C	W	E	E	E	W
W	E	E	E	W	E	W	E	E	W	W	E	W	E	W
W	W	W	E	W	E	W	E	E	E	E	W	H	W	W
W	E	C	E	W	E	W	E	E	W	W	W	W	E	W
W	E	S	W	E	E	C	E	E	E	E	E	C	E	W
W	W	W	W	W	W	W	W	W	W	W	W	W	W	W

```
def GetStr(a2DList, rowindx):
    #Function to return the nominated list in the 2D list
    as a string
    aRow = a2DList[rowindx] #get a row
    aRow = str(aRow) # cast list to string
    aRow = aRow.replace(", ", "")#replace unwanted chars
    with null
    aRow = aRow.replace("'", "")
    aRow = aRow.replace('"', "")
    #ref:https://www.digitalocean.com/community/tutorials/pyt
    hon-remove-character-from-string18
    return aRow.upper()
```

Since the method to convert a Row in a maze definition into a string is used more than once, I have developed it into a function. This function is used in 2 points of my prototype code.

```

#TestGrid.py
#prototype to draw a grid in pygame
#Rosa 31/08/2024

import pygame

def GetStr(a2DList, rowindx):
    #Function to return the nominated list in the 2D list as a string
    aRow = a2DList[rowindx] #get a row
    aRow = str(aRow) # cast list to string
    aRow = aRow.replace(", ", "")#replace unwanted chars with null
    aRow = aRow.replace("['", "")
    aRow = aRow.replace("'", "")
    return aRow.upper()

#define a maze
aMaze = [
    ["W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W"],
    ["W", "E", "C", "E", "E", "E", "E", "E", "S", "E", "C", "W", "C", "W"],
    ["W", "E", "W", "W", "W", "W", "W", "E", "W", "W", "W", "E", "W", "E", "W"],
    ["W", "E", "W", "E", "E", "S", "C", "E", "E", "W", "W", "E", "W", "E", "W"],
    ["W", "E", "W", "E", "W", "E", "W", "W", "E", "C", "W", "E", "E", "E", "W"],
    ["W", "E", "E", "E", "W", "E", "W", "E", "E", "W", "W", "E", "W", "E", "W"],
    ["W", "W", "W", "E", "W", "E", "W", "E", "E", "E", "E", "E", "W", "H", "W"],
    ["W", "E", "C", "E", "W", "E", "W", "E", "E", "W", "W", "W", "W", "E", "W"],
    ["W", "E", "S", "W", "E", "E", "C", "E", "E", "E", "E", "E", "C", "E", "W"],
    ["W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W", "W"]
]

#define colours for different elements
colEmpty = "#FFFFFF"
colHero = "#FF0000"
colSpy = "#4472C4"
colClue = "#ED7D31"
colWall = "#821890"

#to be determined from maze defenition
NumRows = len(aMaze) #calculate number of rows
aRow = GetStr(aMaze, 0)
NumCols = len(aRow) # calculate number of columns

#for a cell
Width = 20 #pixels (square cell)
Thick = 5 #pixels (gap between cells)
#CellColour = pygame.Color("darkturquoise")
#for the background
BackWidth = (NumCols * Width) + ((NumCols+1) * Thick)
BackHeight = (NumRows * Width) + ((NumRows+1) * Thick)
BackColour = pygame.Color("darkslategrey")
#Ref 17: https://www.pygame.org/docs/ref/color\_list.html

```

```

#Set up a pygame window
pygame.init()
pygame.display.set_caption("Prototype_Grid1"
)

#iterate to draw grid
SCREEN = pygame.display.set_mode((BackWidth,
BackHeight))
SCREEN.fill(BackColour)

for Row in range(1, NumRows+1):

    #need to select cell colour for row
    aRow = GetStr(aMaze, Row-1)

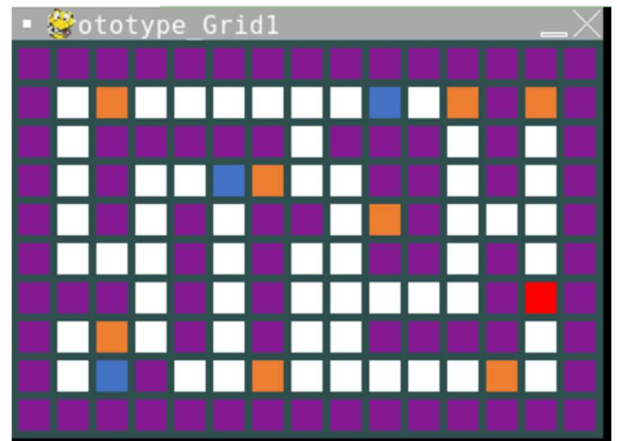
    for Col in range (1, NumCols+1):
        X = (Col)*Thick + (Col-1)*Width
        Y = (Row)*Thick + (Row-1)*Width
        rect = pygame.Rect(X, Y, Width, Width)

        #need to select cell colour
        theChar = aRow[Col-1]
        #selection
        if theChar == "W":
            CellColour = colWall
        elif theChar == "H":
            CellColour = colHero
        elif theChar == "S":
            CellColour = colSpy
        elif theChar == "C":
            CellColour = colClue
        else:
            CellColour = colEmpty

        pygame.draw.rect(SCREEN, CellColour,
rect, 0)

pygame.display.update()
input() #pause

```



Code and output of the Test grid.

36.8. Having used Pygame as shown above to create the maze, I needed it to be drawn from a file so that an admin could input their own mazes. I created these text files for loading it later:

Previously I had to remove square brackets and “,” so I will modify the format of the definition in the file as shown below:

Each row is now just a single string of relevant characters.

```
Maze1.txt
File Edit View
W, W, W, W, W, W, W, W, W, W, W, W, W, W, W
W, E, C, E, E, E, E, E, S, E, C, W, C, W
W, E, W, W, W, W, E, W, W, W, E, W, E, W
W, E, W, E, E, S, C, E, E, W, W, E, W, E, W
W, E, W, E, W, E, W, E, C, W, E, E, E, W
W, E, E, E, W, E, W, E, E, W, W, E, W, E, W
W, W, W, E, W, E, W, E, E, E, E, W, H, W
W, E, C, E, W, E, W, E, E, W, W, W, E, W
W, E, S, W, E, E, C, E, E, E, E, C, E, W
W, W, W, W, W, W, W, W, W, W, W, W, W, W

Maze1.txt
File Edit View
WWWWWWWWWWWWWW
WECEEEEESEWCW
WEWWWWWWWWWEW
WEWEESCEWWWEW
WEWEWWWECWEEW
WEEWEWEWWWEW
WWWWEWEEEEWHW
WECEWEWEWWWEW
WESWEECEEECEW
WWWWWWWWWWWW
```

36.9. **Loading a File.** I developed a function to take a filename and return its contents as a maze.

36.10. The Success criteria for loading a file is:

- A valid Filename has its contents returned
- An invalid Filename has an error message displayed and NULL string returned.

36.11. Design and Development for Loading a File:

```

Function LoadFile(aFile)

    //function to return contents of aFile

    FileReader = openRead(aFile)    //open a file for reading

    If opening aFile fails then

        Print("Error with file")

        Return ""    //return Null char

    Else

        AllLines = [] //empty array

        While not FileReader.EndOfFile()

            AllLines.append(FileReader.readLine())    //add line to the array

        Endwhile

        //get here when all lines are read

        FileReader.close()    //close file when finished

        Return AllLines

    Endif

Endfunction

```

```

def LoadFile(aFile):
    #Function to return contents of aFile
    try:

        #using 'with' closes the file automatically
        with open(aFile) as FileReader:

            AllLines = FileReader.readlines()
            return AllLines

    except:

        print("ERROR with %s" %aFile)
        return ""    #NULL char

```

The Maze definition has been loaded correctly as a list of strings per row.

I have coded the function.

Using exception handling and 'with' to open the file that closes it automatically when finished

```

#load the Maze from the file
aMaze = LoadFile("Mazel.txt")

print(aMaze)    #result?

#to be determined from maze definition
NumRows = len(aMaze) #calculate number of rows
NumCols = len(aMaze[0]) # calculate number of columns

print("NumRows = %i; NumCols = %i" % (NumRows, NumCols))

['WWWWWWWWWWWWWW\n', 'WEEEEESEECWCW\n', 'WEEWWWWWWWWWWWW\n', 'WEEESCEEWW
EWEW\n', 'WEEWWWWWWWWWWWW\n', 'WEEWWWWWWWWWWWW\n', 'WEEWWWWWWWWWWWW\n', 'WEE
EWEWWWWWWWWWW\n', 'WEESEEEEEECEW\n', 'WWWWWWWWWWWWWW\n']
NumRows 10; NumCols = 16

```

Calling the LoadFile function and displaying the values.

But the code has counted 16 columns rather than 15. This is because the newline is being counted in row 1. Either remove the newline when the file is loaded or count the chars in the last row where there is no new line. For now, since I want to re-use the functions to load Clue files, I will remove the newline in LoadFile()

```
def LoadFile(aFile):
    #Function to return contents of aFile
    try:
        #using 'with' closes the file automatically
        with open(aFile) as FileReader:
            AllLines = FileReader.readlines()

            #iterate to remove new lines
            for row in range(len(AllLines)):
                AllLines[row] = AllLines[row].replace("\n", "")
            return AllLines
    except:
        print("ERROR with %s" %aFile)
        return "" #NULL char
```

LoadFile now iterates to remove the new lines

```

Maze from the pygame community: https://www.pygame.org/contributions/maze
['WWWWWWWWWWWWWWWW', 'WEEEEEEEESECWCW', 'WEWWWWWEWWWEWEW', 'WEWEESCEEWWWEWEW'
'WEWEWEWECWEEEW', 'WEEEWEEEWWEWEW', 'WWWEWEWEEEEEEWHW', 'WECEWEWEWEWWWEW',
'WESWEECEEEEEECEW', 'WWWWWWWWWWWWWWWW']
NumRows = 10; NumCols = 15

```

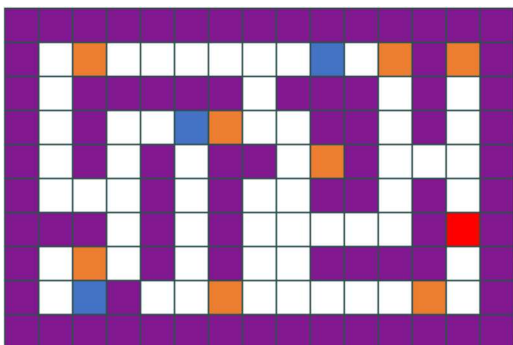
Each row has the new line removed and the correct number of columns is calculated.

This passes the test for working with a valid file

```
#load the Maze from the file
aMaze = LoadFile("NoMaze.txt")
```

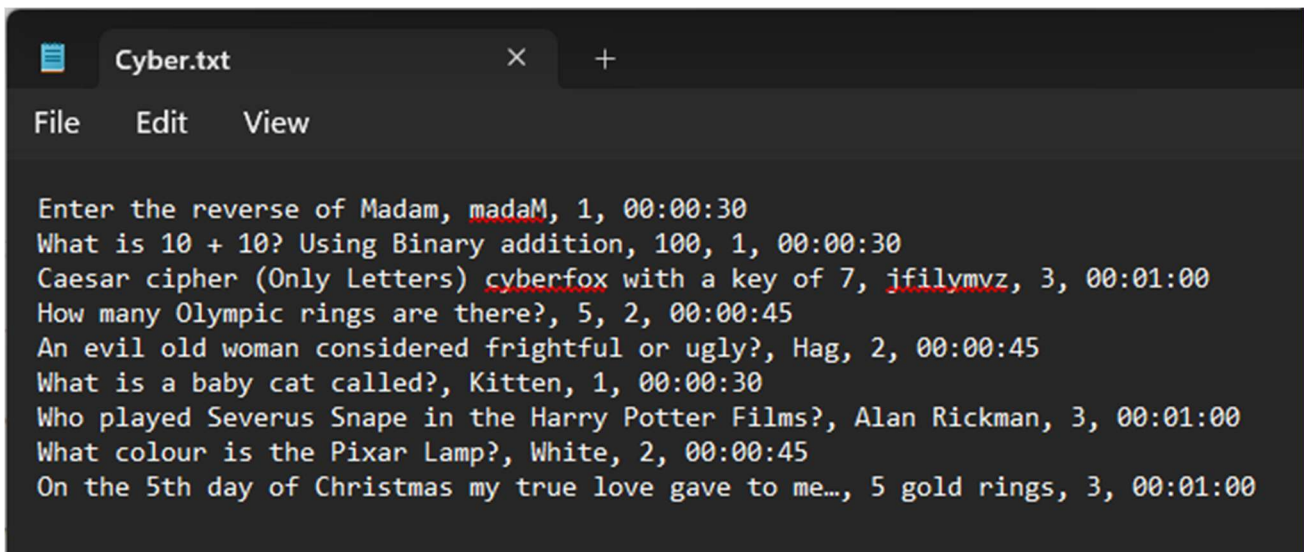
ERROR with NoMaze.txt

Loading a non-existent (invalid)file: **Test passes for managing an invalid file**



The Maze is now drawn correctly as specified in the file

36.12. Extra Test – Loading a Clueset – I need a game to be able to load a maze file and its associated clueset



```
File Edit View

Enter the reverse of Madam, madaM, 1, 00:00:30
What is 10 + 10? Using Binary addition, 100, 1, 00:00:30
Caesar cipher (Only Letters) cyberfox with a key of 7, ifilymvz, 3, 00:01:00
How many Olympic rings are there?, 5, 2, 00:00:45
An evil old woman considered frightful or ugly?, Hag, 2, 00:00:45
What is a baby cat called?, Kitten, 1, 00:00:30
Who played Severus Snape in the Harry Potter Films?, Alan Rickman, 3, 00:01:00
What colour is the Pixar Lamp?, White, 2, 00:00:45
On the 5th day of Christmas my true love gave to me..., 5 gold rings, 3, 00:01:00
```

36.13. This is a Test file with clues and their difficulty and time allowed.

```
>>> Clues = LoadFile("Cyber.txt")
>>> len(Clues)
9
>>> print(Clues[3])

SyntaxError: invalid syntax
>>> print(Clues[3])
How many Olympic rings are there?, 5, 2, 00:00:45
>>> Parts = Clues[3].split(", ")
>>> print("Question = ", Parts[0])
Question = How many Olympic rings are there?
>>>
```

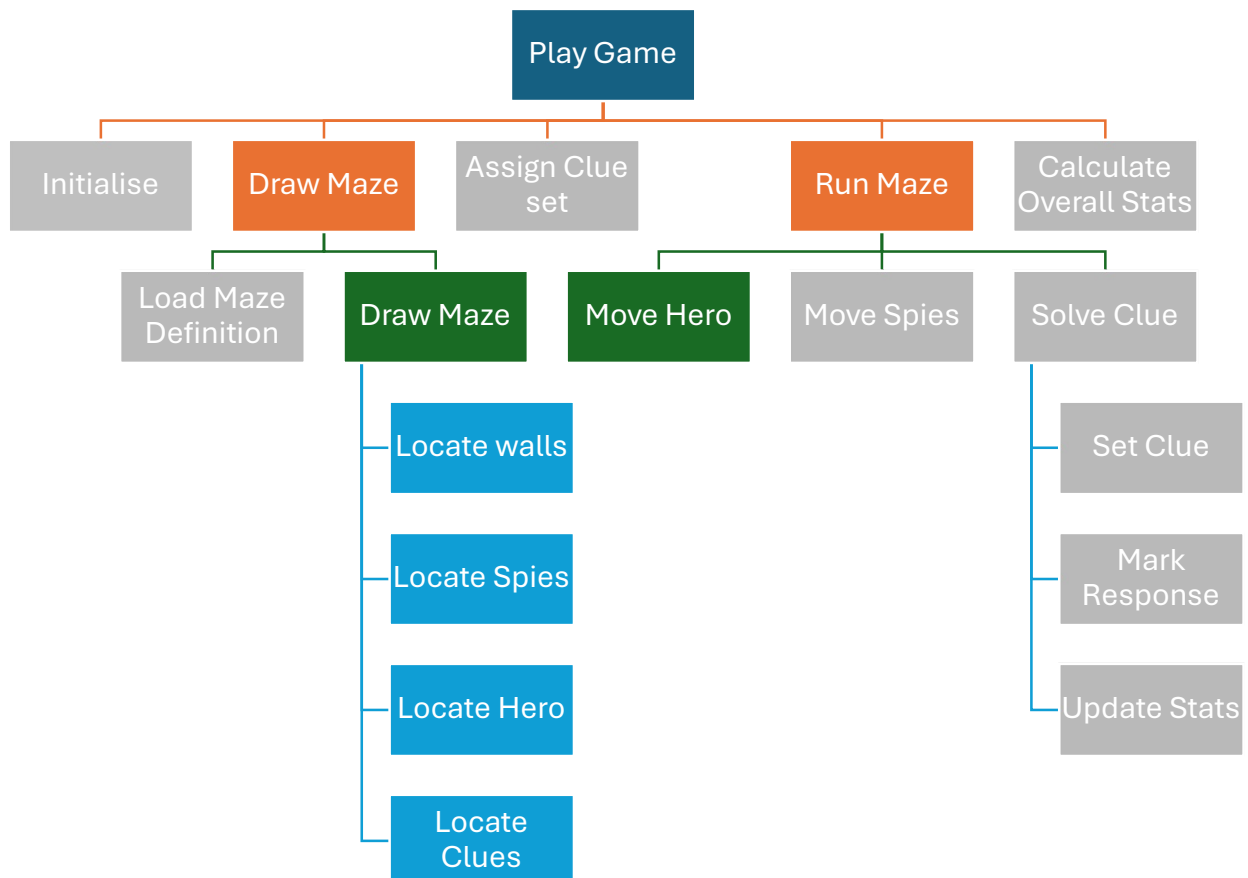
The LoadFile Function loads the file and removes the new lines correctly.

I can index each record, and I can split the selected record into its parts.

36.14. **Evaluation:** The function LoadFile() will correctly load and return the contents of the Maze and Clue text files for use in the Game.

Updating the Maze.

36.15. I am now going to continue to consider the different elements of my decomposed system to update the game to respond to any changes within the gameplay, such as Hero movement. I am focusing on the coloured boxes in the diagram below:



36.16. Now that I can draw a Maze from its definition file, I need to manage how to update it whenever there is a change, such as:

- A Hero Moving
- A Spy moving
- A Clue being removed

To Move the Hero:

- Scan the maze to locate the hero
- Determine:
 - if the move is legal
 - if the move intercepts a clue – so I need to know where the clues are
 - if the move is into a Spy – so I need to know where all the spies are

36.17. A better way than frequently scanning the current maze would be to record the position of each entity at the start and update them. I can use arrays of cell coordinates to do this as follows:

posnHero = [(row, col)] – coordinates of Hero

posnSpy = [(row1, col1), ..., (rown, coln)] - coordinates of all the Spies

posnClue = [(row1, col1), ..., (rown, coln)] - coordinates of all the Clues

posnWall = [(row1, col1), ..., (rown, coln)] - coordinates of all the Walls

Future Development Idea: The walls could potentially move in a future prototype.

36.18. Now the grid can be drawn completely empty at the start, then adding the following in this order:

- the Clues
- the Hero
- the Spy
- the Walls

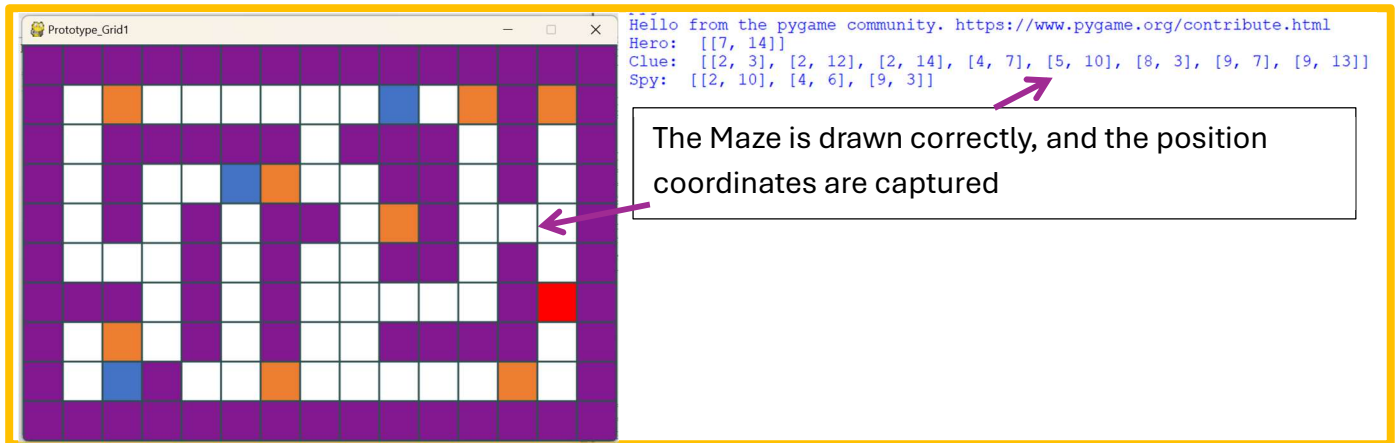
I will modify my development code to generate the posnX arrays:

```
34 # create empty lists for the coordinates of the entities
35 posnHero = []
36 posnSpy = []
37 posnClue = []
38 posnWall = []

80 #need to select cell colour
81 theChar = aRow[Col-1]
82 #selection and find coordinates
83 if theChar == "W":
84     CellColour = colWall
85     posnWall.append([Row, Col])
86
87 elif theChar == "H":
88     CellColour = colHero
89     posnHero.append([Row, Col])
90
91 elif theChar == "S":
92     CellColour = colSpy
93     posnSpy.append([Row, Col])
94
95 elif theChar == "C":
96     CellColour = colClue
97     posnClue.append([Row, Col])
98
99 else:
100     CellColour = colEmpty
```

The code initialises empty list for the cell coordinates.

The cell coordinate (row, col) is added to each entity's position list as it is detected.



Evaluation of Game Operation So Far.

36.19. So far, I have prototyped:

- Reading a maze definition from a text file → created a reusable LoadFile function.
- Creating separate lists of cell coordinates in the maze
- Drawing the maze at the start with the correct cell colours →
Maintenance (achieved for first prototype): currently all characters in my game are coloured square boxes and it would be nice to have icons to represent the characters.
- Drawing the surface (screen) the correct size for the given maze.

Tidying up Ready to Develop the Game Action

36.20. To continue prototyping, I want to be able to draw the maze using the position lists so I need to separate and configure 3 more subprograms:

- **GetStartPosns** – a procedure to create the position lists at the start from the Maze definition
- **DrawMaze** – a procedure to access the global position lists and draw the maze at any time. This can be used throughout the game to update the maze display.
- **Initialise** – a procedure to set global values user throughout the game e.g. Cell colours

GetStartPosns Sub Programme

36.21. This subprogram needs to take a loaded Maze definition, scan it and add cell coordinates to each position list – posnHero, posnSpy, posnClue, posnWall

```
//use globals because they will be used in more than one subprogram
Global posnHero
Global posnSpy
Global posnClue
Global posnWall
Global NumRows
Global NumCols          //maze dimensions – used in more than 1
subprogram
Procedure GetStartPosns(aMaze)
//procedure to populate the global posn lists by scanning the given aMaze
parameter
    NumRows = Maze.length()
    NumCols = Maze[0].length()
    //iterate
    For Row = 0 to NumRows – 1
        For Col = 0 to NumCols – 1
            //selection
            If aMaze[Row, Col].upper() = "W" then
                posnWall.append((Row, Col))
            elseif aMaze[Row, Col].upper() = "H" then
                posnHero.append((Row, Col))
            elseif aMaze[Row, Col].upper() = "C" then
                posnClue.append((Row, Col))
            elseif aMaze[Row, Col].upper() = "S" then
                posnSpy.append((Row, Col))
            endif
            //if none of the above then cell is Empty
        Next Col
    Next Row
EndProcedure
```

```
def GetStartPosns(aMaze):
    #procedure to populate the Global posn
    list by reading aMaze
```

```
    #changing globals can be changed
    global posnWall, posnClue, posnHero,
    posnSpy
```

```
    global NumRows, NumCols    #will be
    created as global variables
```

```
    #get mze dimentions
```

```
    NumRows = len(aMaze)
    NumCols = len(aMaze[0])
```

```
    #draw the grid of cells
```

```
    for Row in range(NumRows):
        Y = (Row - 1)*Thick + (Row)*Width
```

```
    #need to select cell colour for row
    aRow = aMaze[Row] #get a string
```

```
    for Col in range (NumCols):
        X = (Col-1)*Thick + (Col)*Width
```

```
    #need to select cell colour
```

```
    theChar = aRow[Col].upper()
```

```
    #selection and find coordinates
```

```
    if theChar == "W":
        posnWall.append([Row, Col])
```

```
    elif theChar == "H":
        posnHero.append([Row, Col])
```

```
    elif theChar == "S":
        posnSpy.append([Row, Col])
```

```
    elif theChar == "C":
        posnClue.append([Row, Col])
```

```
    return #endprocedure
```

```
156 print("\nTest of GetStartPosns procedure: ")
157 print("NumRows: ", NumRows)
158 print("NumCols: ", NumCols)
159 print("Hero: ", posnHero)
160 print("Clue: ", posnClue)
161 print("Spy: ", posnSpy)
162 print("Wall: ", posnWall)
```

From main call GetStartPosns() and display globals

The Procedure correctly sets the global values

Test of GetStartPosns procedure:

NumRows: 10

NumCols: 15

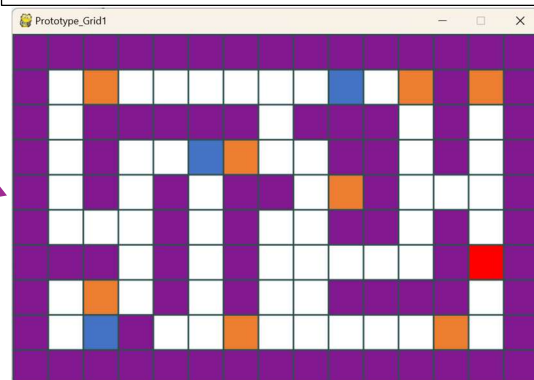
Hero: [[6, 13]]

Clue: [[1, 2], [1, 11], [1, 13], [3, 6], [4, 9], [7, 2], [8, 6], [8, 12]]

Spy: [[1, 9], [3, 5], [8, 2]]

Wall: [[0, 0], [0, 1], [0, 2], [0, 3], [0, 4], [0, 5], [0, 6], [0, 7], [0, 8], [0, 9], [0, 10], [0, 11], [0, 12], [0, 13], [0, 14], [1, 0], [1, 12], [1, 14], [2, 0], [2, 2], [2, 3], [2, 4], [2, 5], [2, 6], [2, 8], [2, 9], [2, 10], [2, 12], [2, 14], [3, 0], [3, 2], [3, 9], [3, 10], [3, 12], [3, 14], [4, 0], [4, 2], [4, 4], [4, 6], [4, 7], [4, 10], [4, 14], [5, 0], [5, 4], [5, 6], [5, 9], [5, 10], [5, 12], [5, 14], [6, 0], [6, 1], [6, 2], [6, 4], [6, 6], [6, 12], [6, 14], [7, 0], [7, 4], [7, 6], [7, 9], [7, 10], [7, 11], [7, 12], [7, 14], [8, 0], [8, 3], [8, 14], [9, 0], [9, 1], [9, 2], [9, 3], [9, 4], [9, 5], [9, 6], [9, 7], [9, 8], [9, 9], [9, 10], [9, 11], [9, 12], [9, 13], [9, 14]]

The Posn Lists correctly match the maze definition



DrawMaze Sub Programme

36.22. This procedure reads the global posn lists and NumRows, NumCols and draws the maze. This will be used to update the maze as the game progresses and the Hero and Spies move and the clues are removed.

```
Procedure DrawMaze()
```

```
    //procedure to draw maze using global variables and lists
```

```
    //apply the precedence if 2 entities share the same space
```

```
    //iterate
```

```
    For Row = 0 to NumRows-1
```

```
        For Col = 0 to NumCols-1
```

```
            CellCoord= (Row, Col)
```

```
            If CellCoord in posnWall then
```

```
                cellColour = colWall
```

```
            elseif CellCoord in posnSpy then
```

```
                cellColour = colSpy
```

```
            elseif CellCoord in posnHero then
```

```
                cellColour = colHero
```

```
            elseif CellCoord in posnClue then
```

```
                cellColour = colClue
```

```
            else    //no entity in this cell
```

```
                cellColour = colEmpty
```

```
            endif
```

```
            draw cell at CellCoord with cellColour
```

```
        next Col
```

```
    next Row
```

```
endprocedure
```

```

def DrawMaze():
    #draw the grid of cells
    global SCREEN #because procedure modifies it
    for Row in range(1, NumRows+1):
        Y = (Row)*Thick + (Row-1)*Width

        for Col in range (1, NumCols+1):
            X = (Col)*Thick + (Col-1)*Width

            rect = pygame.Rect(X, Y, Width, Width)

            #need to select cell colour according
            #to precedence
            cellCoord = (Row-1, Col-1) #tuple for
            cell coordinate
            #selection and find coordinate
            if cellCoord in posnWall:
                CellColour = colWall
            elif cellCoord in posnSpy:
                CellColour = colSpy

            elif cellCoord in posnClue:
                CellColour = colClue

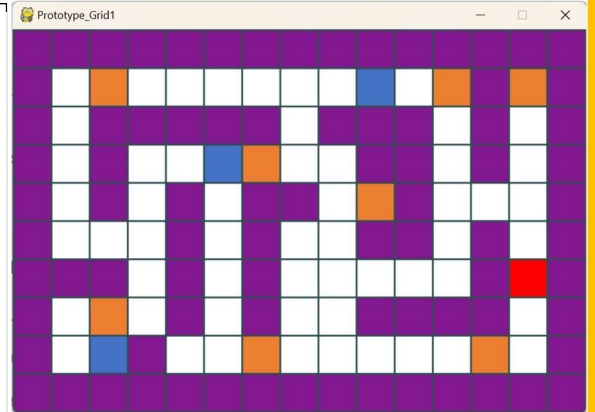
            elif cellCoord in posnHero:
                CellColour = colHero

            else:
                CellColour = colEmpty

            pygame.draw.rect(SCREEN, CellColour,
            rect, 0)

        pygame.display.update()
    return

```



The Maze is drawn correctly using the global values.

```

155 #draw grid of cells
156 DrawMaze() #call procedure
157 print("Hero: ", posnHero)
158 #change some position values
159 posnHero = [(3,6)]
160 #display new postitons
161 print("Hero: ", posnHero)
162 #update view of maze
163 DrawMaze()
164

```

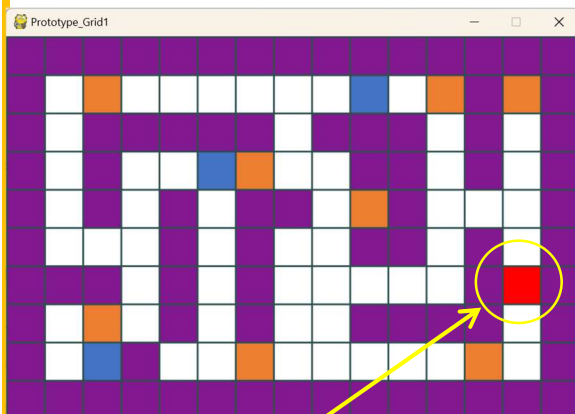
To test DrawMaze() works with updated positions.

- Start with start positions

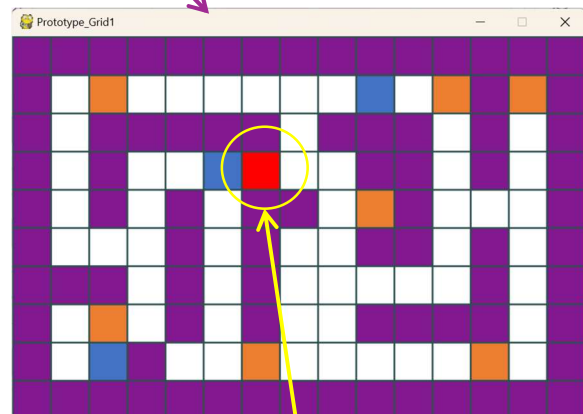
```

Hero: [(6, 13)]
Hero: [(3, 6)]
>>> |

```



Hero Start position



Hero Updated position. Hero has now been redrawn in the new position.

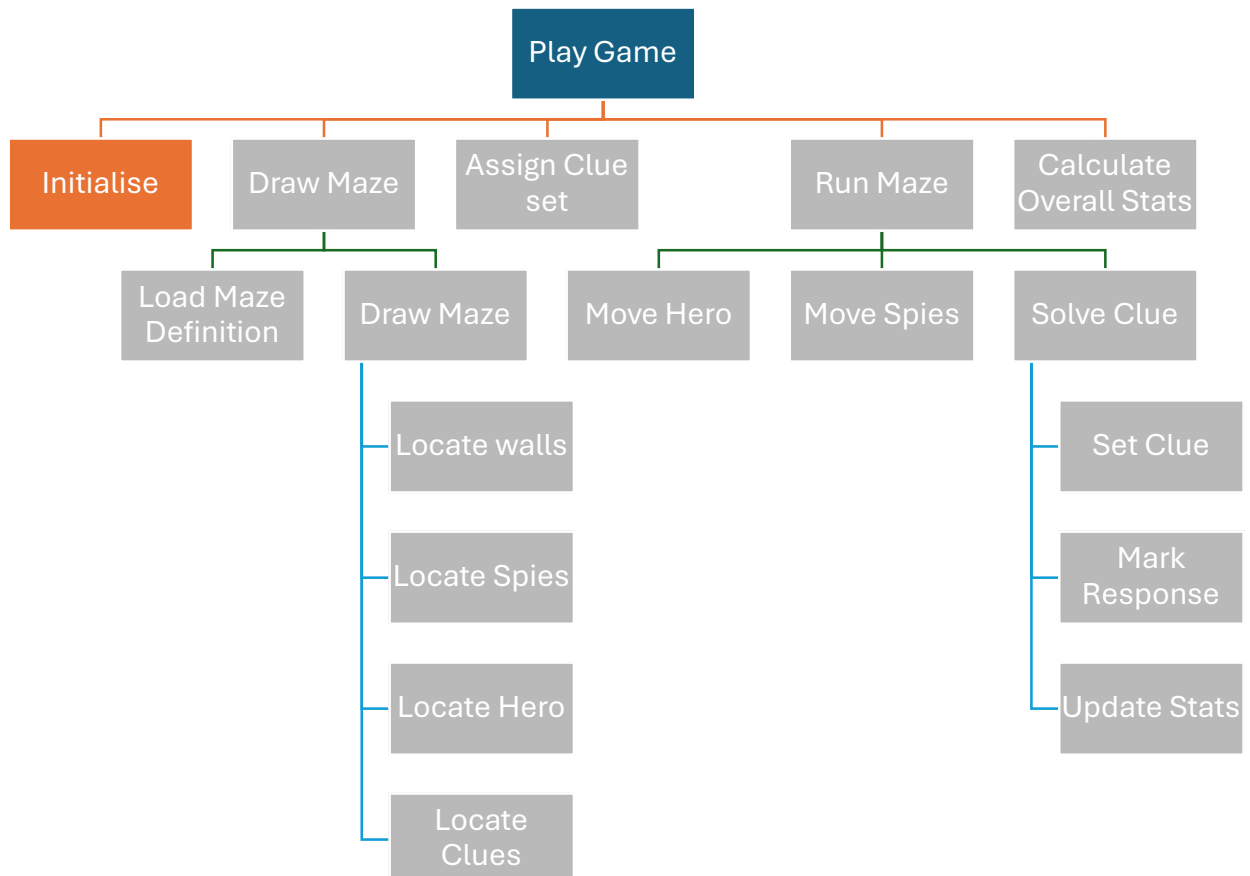
Test that DrawMaze works with updated positions by moving the hero.

Evaluation of Draw Maze:

36.23. The DrawMaze() Procedure works with the position list, however they have been changed. This will be critical when refreshing the display up to 30 fps!

Initialise Sub Programme

36.24. I am now going to continue to consider the different elements of my decomposed system to fully develop my Initialise function in its entirety, as shown in the coloured box below:



36.25. I am taking the code that I have developed to set up the drawing of the maze and I am putting it in its own procedure, declaring global variables to use elsewhere.

```

31 def Initialise():
32     #procedure to identify global variables and set their start values
33     #define global colours
34     global colEmpty, colHero, colSpy, colClue, colWall, colBG
35
36     #define colours for different elements - note: replace with Avatars
37     colEmpty = (255, 255, 255)      #rgb colours instead of hex tuples
38     colHero = (255, 0, 0)
39     colSpy = (68, 114, 196)
40     colClue = (237, 125, 49)
41     colWall = (130, 24, 144)
42     #ref: https://www.rapidtables.com/convert/color/hex-to-rgb.html19
43     colBG = "darkslategrey"      #Background colour of maze
44
45     #global maze dimension
46     global Width, Thick
47
48     #define maze dimensions
49     #for a cell
50     Width = 40 #pixels (square cell)
51     Thick = 2 #pixels (gap between cells)
52
53     #create global empty lists for the coordinates of the entities
54     global posnHero, posnSpy, posnClue, posnWall
55
56     # create empty lists for the coordinates of the entities
57     posnHero = []
58     posnSpy = []
59     posnClue = []
60     posnWall = []
61
62     return #end procedure
63

```

The Initialise() procedure declares global variables that can be easily accessed and changed by other subprograms, and sets their starting values

```

8 def TestInitialise():
9     #procedure to test initialisation
10    Initialise() #call the procedure
11    print("Test of Initialisation")
12    print("\nColours: ")
13    print("\tcolEmpty: %s"%str(colEmpty))
14    print("\tcolHero: %s"%str(colHero))
15    print("\tcolSpy: %s"%str(colSpy))
16    print("\tcolWall: %s"%str(colWall))
17    print("\tcolClue: %s"%str(colClue))
18    print("\tcolBG: %s"%str(colBG))
19    print("\nCell dimensions:")
20    print("\nCell width/height: %s px"%str(Width))
21    print("\tCell Border: %s px"%str(Thick))
22    print("\nPosition lists")
23    print("\tHero =", posnHero)
24    print("\tSpy =", posnSpy)
25    print("\tClue =", posnClue)
26    print("\tWall =", posnWall)
27
28    return #end procedure

```

Test Initialise

I wrote a procedure to test the Initialise() procedure that can only access global variables and output their values.

The Global variables are accessed and the start values are correct

TEST PASSED

Evaluation: This was an important test to check that the initialise procedure of the game was set up correctly. As it is working properly, I can also use this procedure to be called at the startup of the game within an overall function.

Test of Initialisation

Colours:

```

colEmpty: (255, 255, 255)
colHero: (255, 0, 0)
colSpy: (68, 114, 196)
colWall: (130, 24, 144)
colClue: (237, 125, 49)
colBG: darkslategrey

```

Cell dimensions:

```

Cell width/height: 40 px
Cell Border: 2 px

```

Position lists

```

Hero = []
Spy = []
Clue = []
Wall = []

```

Evaluation of Initialise Sub Programme:

36.26. The basic values are set, but can the maze be loaded and drawn as part of the initialisation?

```

65 #set up the start maze configuration
66
67 #load the Maze from the file
68 aMaze = LoadFile(aMazeFile)
69
70 #populate the posn lists and Global NumRows, NumCols
71 GetStartPosns(aMaze) #call procedure
72
73 #Draw the maze as a grid
74 #for the background
75 BackWidth = (NumCols * Width) + ((NumCols+1) * Thick)
76 BackHeight = (NumRows * Width) + ((NumRows+1) * Thick)
77 BackColour = pygame.Color(colBG)
78 #ref: https://www.pygame.org/docs/ref/color_list.html17
79
80 #Set up a pygame window
81 pygame.init()
82 pygame.display.set_caption("Prototype_Grid2")
83
84 #iterate to draw grid
85 global SCREEN # for game updates as progresses
86 SCREEN = pygame.display.set_mode((BackWidth, BackHeight))
87 SCREEN.fill(BackColour)
88
89 #Draw grid
90 DrawMaze() #call procedure
91
187 #####MAIN#####
188
189 #set up global values and draw maze at start
190 Initialise("Maze1.txt")
191
192 #change position values
193 posnHero = [(3,6)]
194
195 #update view of maze
196 DrawMaze()

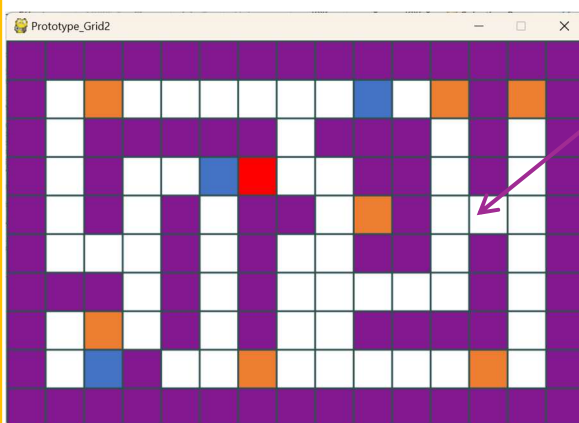
```

I have moved the code from main into the Initialise procedure to draw the maze at the start of the game, setting and using the global.

The Maze filename is now passed as a parameter to initialise().

The Main is much simplified – the maze filename is passed as a parameter in the call Initialise.

DrawMaze is called independently to update the view of the maze after moving the Hero



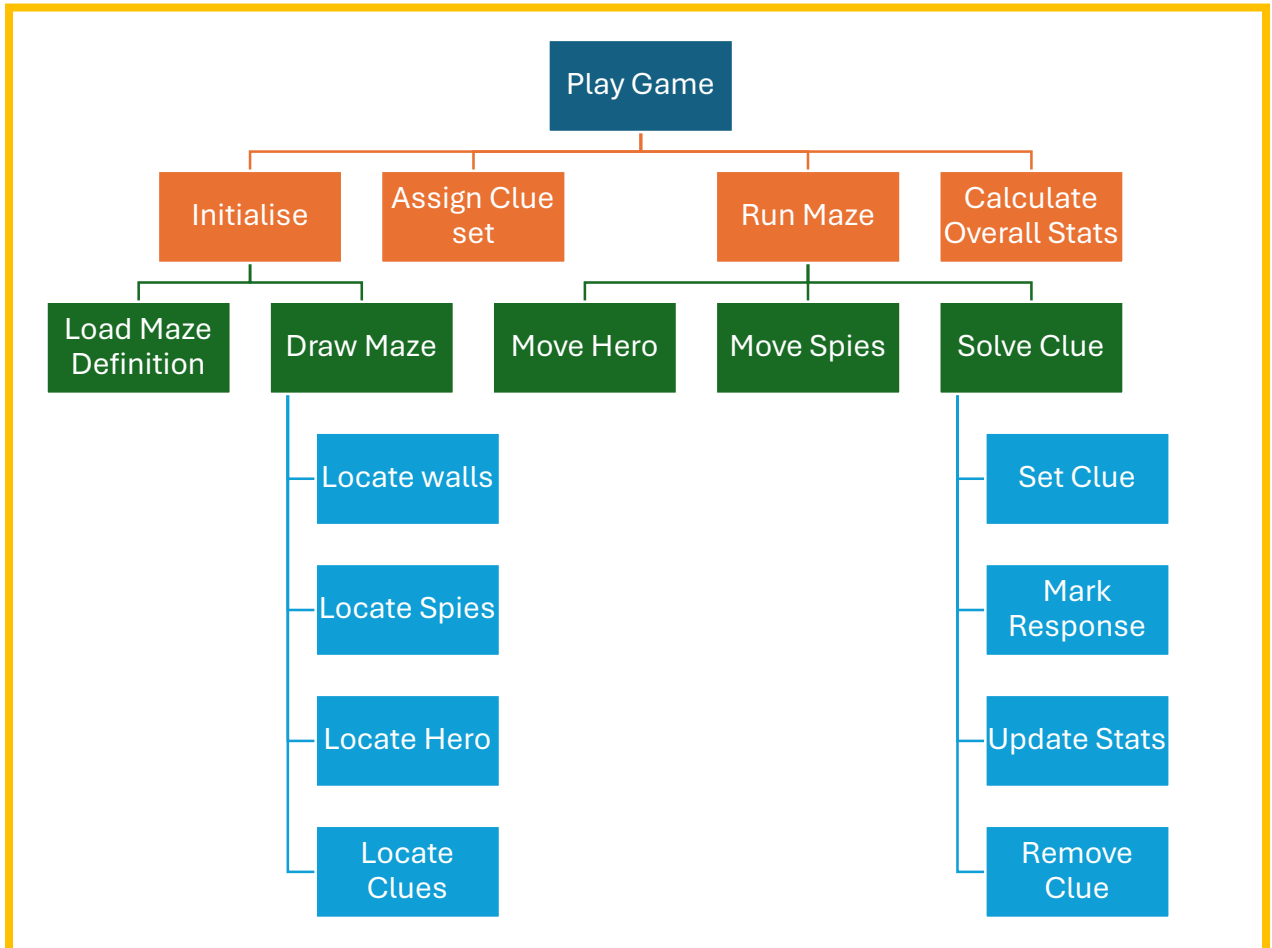
The maze has been drawn correctly
DrawMaze accesses the global position list correctly

Evaluation of Setting up Default Values for Maze:

36.27. I have created the ability to manage setting up the default values for the maze and using DrawMaze() to update it, based on a given maze definition file.

Playing the Game

36.28. My original decomposition for the game has been modified slightly and the 'Create Maze' on my original decomposition is now included within the 'Initialise' part of the decomposition as 'Draw Maze' as follows:

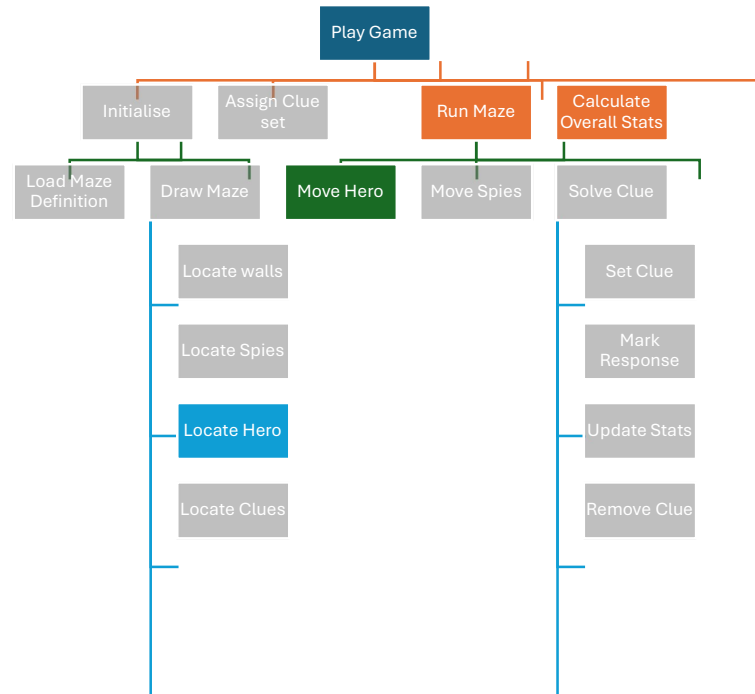


36.29. I have found this reference <https://realpython.com/pygame-a-primer/>¹¹ for creating a game using PyGame and have drawn on it to structure my game development. I have developed the following Algorithm for playing a maze game:

Initialise – set up the globals and draw the start maze with objects in their start positions
While not Game Over
 Handle any keyboard events – arrow keys for hero , quit to close game window
 Move any objects under program control -Spies
 Update any game stats – Score, Lives, Time Taken
 Redraw the maze/refresh the maze 30fps (frames per second) for animation
Endwhile
Return game stats

Moving the Hero Using Key Presses

36.30. I am now going to continue to consider the different elements of my new decomposed system to move the Hero within my game using key presses. This is shown by the coloured boxes on the diagram below:



A. Moving The Hero

1. The Hero is moved by the Player pressing the arrow keys
 - a. Up arrow key → up (y-1)
 - b. Down arrow key → down (y+1)
 - c. Right arrow key → right (x+1)
 - d. Left arrow key → left (x-1)
2. Precedence: when moved, the hero
 - a. The hero moves under a spy
 - b. The hero moves over a clue
 - c. The hero cannot move into a wall (move not allowed)
3. If the game window "X" is clicked close the window and Quit.

B. GameOver

- It is game over and the game loop stops when:
 - The Max time for the game has elapsed
 - The Lives = 0
 - There are no more clues to solve.
- When it is game over the game stats are returned for recording in the database.
 - Score
 - Time taken
 - Lives Left (cannot be 0 – game not completed)

36.31. The algorithm I am using to move the hero is as follows:

```
If an arrow key press is detected then:
    Currentposn = posnHero[0]    //Get the current hero position (Row,Col)
    CurrentX = Currentposn[1]
    CurrentY = Currentposn[0]
    If it is the Up arrow then:
        Newposn = (CurrentY - 1, Currentposn)
    ElseIf it is the Down arrow then:
        Newposn = (CurrentY + 1, Currentposn)
    ElseIf it is the Right arrow then:
        Newposn = (Currentposn, CurrentX + 1)
    ElseIf it is the Left arrow then:
        Newposn = (Currentposn, CurrentX - 1)
    Endif
    //test the impact of the move
    If Newposn in posnWall then          //move is illegal
        posnHero = CurrentPosn          //no move this event
    ElseIf Newposn in posnClues then      //no move this time
        //needs further clarification
        //pause game timer
        //select and display clue
        //mark the answer – increase Score if correct
        Remove New Posn from posnClues    //delete clue whether
correct or not.
        If posnClues is empty then
            GameOver = True              //stop the game loop
        endif
    ElseIf collided with spy
        Loose a life
        If lives == 0
            GameOver = True //stop the game loop
        Endif
        //at 30 fps, if the hero and the spy share the same cell, all lives will be
lost and it will be game over very quickly.
        //Possible options:
        1. Return Hero and/or Spies to respective original start positions
           as they will not be touching.
        2. Put Hero and/or spies in a random position not in the wall! Take
           the risk the hero could still be touching a Spy. Unless the new
           positions are checked and recalculated to avoid touching and
           placing the Hero on an existing clue
        3. Put hero into available empty cell no wall, clue or spy
           occupying it.
    Endif.
```

Reminder: I need to ask stakeholders
for feedback on highlighted areas

36.32. As highlighted above, I sought addition stakeholder feedback to direct my project and to provide feedback on my 3 options as follows:

Stakeholder Feedback and the Conclusions this lead to:

For the Spy collisions

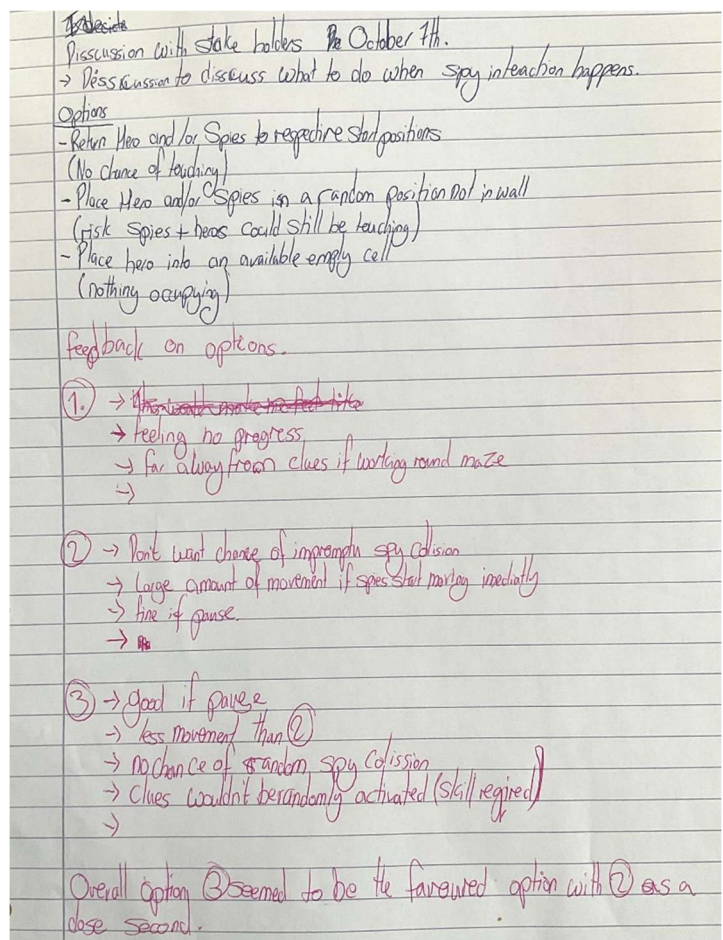
Overall from my stakeholder meeting (notes from this meeting are shown below) It was clear that Option 1 was thought to make progress feel limited. Due to this, Options 2 and 3 were the favoured options. Option 3 was the most favourable due to the reduced risk of random spy collisions. The Stakeholders also responded with the idea of a pause for the User to readjust before starting the spy movements again.

Note: I prefer to pursue Option 3 – moving just the hero which would require:

- Creating a posnEmpty list when the maze is initialised
- Every time the hero moves successfully, the new position is removed from posnEmpty and the previous position is added.
- Every time a spy moves successfully, the posnEmpty array must be updated, removing the new position and adding the old position.
- Every time a clue is removed its position is added to the posnEmpty when the hero leaves the position. (otherwise I will have an empty cell matching the hero position).

The options are:

1. Every time a hero or spy moves successfully, remove the new position from posnEmpty and replace with the new position
2. After 1 iteration of Hero and Spy moves, including removing a Clue, scan the posnHero, posnSpy and posnClue lists and construct a new posnEmpty.



Working Key Presses.

36.33. The following is my coding for the working key press algorithm shown above:

```
# Import pygame.locals for easier access to key coordinates
from pygame.locals import (
    K_UP,
    K_DOWN,
    K_LEFT,
    K_RIGHT,
    KEYDOWN,
    QUIT,
)
```

Importing key functions from the pygame library to be used to move the Hero.

Using the keypress functions to move the hero.

```
UP
    BEFORE: row: 6  col: 13
    AFTER:  row: 5  col: 13
```

Output from "up" keypress

```
# Was it an arrow key to move the Hero?
if event.key == K_UP:
    print("UP") #testing

    yHero, xHero = posnHero[0] # saved position as (row, col)

    print("\tBEFORE:\trow: %i\tcol: %i" %(yHero, xHero))

    yHero = yHero - 1 # move up

    posnHero[0] = (yHero, xHero) # update position

    print("\tAFTER:\trow: %i\tcol: %i\n" %(yHero, xHero))
```

Creating elifs for the other arrow key presses.

```
elif event.key == K_DOWN:
elif event.key == K_RIGHT:
elif event.key == K_LEFT:
```

elif statements using the imported pygame keypress functions

```
yHero = yHero + 1 # move down
posnHero[0] = (yHero, xHero) # update position
xHero = xHero + 1 # move right
posnHero[0] = (yHero, xHero) # update position
xHero = xHero - 1 # move up
posnHero[0] = (yHero, xHero) # update position
```

Changing the Up instructions for the other arrow key presses includes changing the change in y or x dependent on the direction the hero should go.

Working Hero/Spy interaction:

36.34. This sends the hero to another location not occupied by any other entities.

```
posnEmpty = []
```

Added a posnEmpty array to keep track of the empty cells.

Appends the posnEmpty array to keep it up to date editing the array whenever the hero is moved.

```
currentposn = (currenty, currentx)
posnEmpty.append(currentposn)
```

If statement to remove the new position of the hero from the posnEmpty array.

```
elif NewPosn in posnEmpty:
    posnEmpty.remove(NewPosn)
    print(posnEmpty)
```

LEFT

```
BEFORE: row: 1 col: 11
[(1, 1), (1, 3), (1, 4), (1, 5), (1, 6), (2, 1), (3, 1), (3, 3), (3, 4), (4, 1),
 (4, 3), (5, 1), (5, 2), (5, 3), (5, 7), (5, 11), (6, 3), (6, 10), (6, 11), (7,
 1), (7, 3), (8, 1), (8, 4), (8, 5), (8, 7), (7, 8), (6, 8), (5, 8), (4, 8), (3,
 8), (3, 7), (2, 7), (1, 7), (8, 5), (7, 5), (6, 5), (5, 5), (5, 5), (5, 5), (4,
 5), (2, 13), (5, 13), (4, 11), (3, 11), (2, 11)]
AFTER: row: 1 col: 10
```

Printed out posnEmpty array. No coordinates are repeated.

I used a Stackoverflow reference²⁰ to learn how to use the random module in order to randomly select an item from an array to help with this part of the project.

```
import random
posnHero[0] = random.choice(posnEmpty)
```

Import random library allow random.choice to be used to sent the Hero to a random position not occupied by another entity.

```
yHero, xHero = posnHero[0]
NewPosn = (yHero, xHero)
posnEmpty.remove(NewPosn)
```

Makes sure that the new position of the Hero is removed from posnEmpty. Updates Newposn to remove this coordinate set

Logic Error found

UP

```
BEFORE: row: 4 col: 13
[(1, 1), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (1, 8), (1, 10), (2, 1), (2, 7),
(2, 11), (2, 13), (3, 1), (3, 3), (3, 4), (3, 7), (3, 8), (3, 11), (4, 1), (4,
3), (4, 5), (4, 8), (4, 11), (4, 12), (5, 1), (5, 2), (5, 3), (5, 5), (5, 7), (
5, 8), (5, 11), (6, 3), (6, 5), (6, 7), (6, 8), (6, 9), (6, 10), (6, 11), (7, 1)
, (7, 3), (7, 5), (7, 7), (7, 8), (7, 13), (8, 1), (8, 4), (8, 5), (8, 7), (8, 8
), (8, 9), (8, 10), (8, 11), (8, 13), (6, 13), (5, 13), (4, 13), (4, 13)]
AFTER: row: 3 col: 13
```

When the hero hits a wall and then moves up, a duplicate position is added to the posnEmpty array. This is because the position of currentposn is being added to posnEmpty, even if posnHero hasn't changed. This is creating a duplicate when the hero moves again and a logic error happens before the hero moves, after a wall collision.

```
if NewPosn in posnWall:
    print("Hero hit wall")

    #Hit a wall move not allowed
    posnHero[0] = (currenty, curre
yHero, xHero = posnHero[0]
    posnEmpty.remove(currentposn)
```

To fix this I have removed the currentposn from posnEmpty in the wall collision if statement.

36.35. Adding Stat Values to the Game

```
global Lives, Score, TimeTaken
```

This is the set up for “Game Over”, which happens as soon as the Hero touches a Spy for the maximum number of times (lives) allowed within the game.

I made these variables Global so that they could be used in the PlayGame() function, after being declared in the Initialise function()

```
Lives = Lives - 1  
if Lives == 0:  
    GameOver = True
```

```
if ClueAns() is True:  
    Score = Score + 1  
    print(Score)  
elif (ClueAns() is False) and (Score > 0):  
    Score = Score - 1  
    print(Score)
```

This is the set up for indenting the score when the Hero touches the spy.

The ClueAns function is simplified into a user input currently for testing purposes.

```
def ClueAns():  
    ClueOutcome = bool(input("Is the answer correct True/False > "))  
    if ClueOutcome is True:  
        return True  
    elif False:  
        return False
```


36.36. Test Plan for moving Hero

Note: The test is starting from where the hero is located at the Start of the Game, Hero position (6, 13) (Row, Col):

Due to the wall layout of the maze, the arrow tests need to be performed in order (Up, Up, Left, Right, Down, Quit)

Due to the wall layout of the maze, the wall tests need to be performed in order (Up, Right, Left, Up, Left, Up, Down)

Test No.	Description	Input Data	Expected Outcome	Actual Outcome	P/F
1	Up Arrow – Hero Moves Up	Currentposn = (6, 13)	Newposn = (5, 13)	UP BEFORE: row: 6 col: 13 AFTER: row: 5 col: 13	P
2	Down Arrow – Hero Moves Down	Currentposn = (4, 13)	Newposn = (5, 13)	DOWN BEFORE: row: 4 col: 13 AFTER: row: 5 col: 13	P
3	Right Arrow – Hero Moves Right	Currentposn = (4, 12)	Newposn = (4, 13)	RIGHT BEFORE: row: 4 col: 12 AFTER: row: 4 col: 13	P
4	Left Arrow – Hero moves Left	Currentposn = (4, 13)	Newposn = (4, 12)	LEFT BEFORE: row: 4 col: 13 AFTER: row: 4 col: 12	P
5	Game window closes when “X” is clicked	Click on window close icon “X”	Observe window closes. Quit is reported	QUIT Window is closed	P
6	Hero Moves Up into a wall	Move Hero to Currentposn = (4,12)	Not allowed Newposn = Currentposn Unchanged	UP BEFORE: row: 4 col: 12 Hero hit wall AFTER: row: 4 col: 12	P
7	Hero Moves Down into a wall	Move Hero to Currentposn = (4,12)	Not allowed Newposn = Currentposn Unchanged	DOWN BEFORE: row: 4 col: 12 Hero hit wall AFTER: row: 4 col: 12	P
8	Hero Moves Right into a wall	Move Hero to Currentposn = (5, 13)	Not allowed Newposn = Currentposn Unchanged	RIGHT BEFORE: row: 5 col: 13 Hero hit wall AFTER: row: 5 col: 13	P
9	Hero Moves Left into a wall	Move Hero to Currentposn = (5, 13)	Not allowed Newposn = Currentposn	LEFT BEFORE: row: 5 col: 13 Hero hit wall AFTER: row: 5 col: 13	P

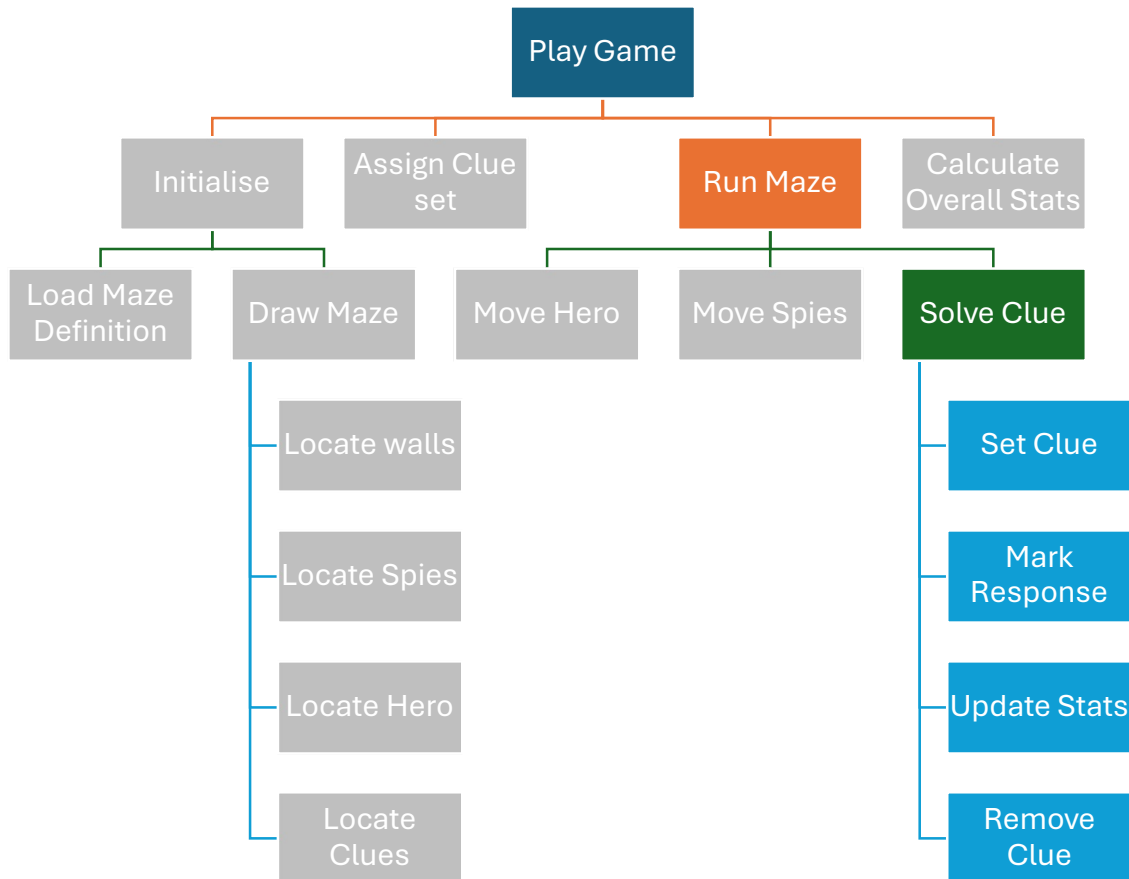
			Unchanged		
10	Hero Moves into Clue (Clue is removed) Partial test for position only	Move Hero to Currentposn = (1,13)	Allowed Newposn = () Posn removed from posnClue and Clue not displayed	UP BEFORE: row: 2 col: 13 Hero hit a Clue AFTER: row: 1 col: 13 	P
11	Hero Moves into Spy Partial test for position only – Hero should be sent to a random unoccupied position in the maze.	Move Hero to Currentposn = (1,8)	Allowed Newposn = (randomly occupied (5,1)) Observe Hero goes to a random unoccupied position after interacting with the Spy. Newposn is in the posnEmpty list and no other lists.	RIGHT BEFORE: row: 1 col: 8 Hero hit a Spy AFTER: row: 1 col: 8 Error: Hero should not be able to be sent to the same position. RIGHT BEFORE: row: 1 col: 8 Hero hit a Spy AFTER: row: 5 col: 1	P
12	Hero Moves into empty cell from any direction	Move Hero to Currentposn = ()	Allowed Newposn = () No conciquences	UP BEFORE: row: 6 col: 13 AFTER: row: 5 col: 13	P

*11. – Test 11 created an error and to fix this I have removed the current position from posnEmpty before randomising, then re-adding it again afterwards.

```
posnEmpty.remove(currentposn)
#ref:https://stackoverflow.com/questions/306
posnHero[0] = random.choice(posnEmpty) #tel
posnEmpty.append(currentposn)
```

Ensuring the hero cannot
land on the same spot.

36.37. I am now going to continue to consider the different elements of my decomposed system to implement the questions within my game. I will be focusing on the areas in the coloured boxes on the diagram below:



36.38. Pausing the Game

I have used an if statement within the while statement for Game Over to allow me to pause the game momentarily to allow the User to answer the Clue questions.

```
if not paused:
    # ref https://r
    # Look at every
    for event in py

    # Did the u
    if event.ty

        current

elif paused:
    #paused
    print(aClue)
    if Lives == 0:
        GameOver = True
    if not GameOver:
```

During these pauses I can answer a question which can be checked against the answer and give the required amount of time per question.

36.39. Adding Questions

I used this code to load the Clue File containing the Clues and it is influenced by the LoadMazeFile function that I have previously created to load in the maze file.

```
def LoadClueFile(Clue):
    Clues = LoadFile(Clue)
    length = len(Clues)
    count = 0
    Parts = []
    for count in range(0, length):
        Parts.append(Clues[count].split(", "))
        count = count+1
    return Parts
```

The code above loads in the Clue File and then splits each question into a separate array along with its answer and other parts. These arrays are all stored within the Parts array. This ensures that I can display only the question whilst also being able to compare the output to the Answer saved in the array. It makes it simpler and easier to access parts of the questions using the indexes.

36.40. Identifying if the User Input Answer is Correct

I have started by generating a random question and having it displayed in the console. An input will then be provided by the User giving an answer to the question. To compare both the answer and the input for matching, first I will change both of them to upper case to avoid case sensitivity, then I will compare the input with the correct answer. If they are equal, then the User's answer is correct.

```
def Question(QuestionFull):  
    DiffQs.append(int(QuestionFull[2]))  
    print(DiffQs)  
    return
```

A short subprogram to store the difficulty of questions for later on down the line.

```
AnsQuestion = random.choice(aClue)  
QuestionFull = AnsQuestion  
print(QuestionFull)  
print(QuestionFull[0])  
Answer = str(input("> "))  
if str(QuestionFull.upper) == Answer.upper:  
    #print statements to test  
    print("True")  
    print(QuestionFull.upper)  
    print(Answer.upper)  
    return True  
elif str(QuestionFull.upper) != Answer.upper:  
    #print statements to test  
    print(QuestionFull.upper)  
    print(Answer.upper)  
    print("False")  
    return False
```

Process of choosing a question from a Clue Set and get an input from the User (this will be in a text box in the future)

Logic error: I forgot that I needed to put brackets at the end of the .upper to make it work so all of the answers were coming back false at first

36.41. Next step: Ask questions based on difficulty (precedence)

This is my first attempt at an Algorithm to ask questions based on the previous question answered by the User so that the User receives questions that are tailored to their level of difficulty:

```
In Initialise create arrays of Easy, Medium and Hard Questions

If DiffQs == [] and LastQ == [] then    //for first question Only
    AnsQuestion = random.choice(aClue)
    Find the clue in the difficulty array and delete it.

Else if not LastQ - 1 then:

    If LastQ[difficulty] == 3 then
        AnsQuestion = Random select from medium array
        Delete the clue in the difficulty array.

    Else if int(LastQ[difficulty]) in range(1,2) then
        AnsQuestion = Random select from easy array
        Delete the clue in the difficulty array.

Else if DiffQs - 1 == 3 OR (EasyQs == [] AND MediumQs == []) OR (MediumQs == [] then)
    AnsQuestion = Random select from hard array
    Delete the clue in the difficulty array.

Else if DiffQs - 1 == 2 OR HardQs == [] OR EasyQs == [] then
    AnsQuestion = Random select from medium array
    Delete the clue in the difficulty array.

Else if DiffQs - 1 == 1 OR (HardQs == [] AND MediumQs == []) then
    AnsQuestion = Random select from array
    Delete the clue in the difficulty array.

Endif
```

36.42. This is my coding of the algorithm above, including a short procedure to split the questions into difficulty level arrays:

```
#declared in initialise
global Num, EasyQs, MedQs, HardQs, LastQ
if Num == 0:
    Ans = random.choice(aClue)
    Full = Ans
    if Full[2] == 3:      #define first Q
        HardQs.remove(Full)
        print("removed hard")
    elif Full[2] == 2:
        MedQs.remove(Full)
        print("removed medium")
    elif Full[2] == 1:
        EasyQs.remove(Full)
        print("removed easy")
    else:
        print("Ahhhhh its not working")
    Num = Num + 1
    LastQ = [Ans[2]]
elif not LastQ[-1]: #boolean
    if LastQ[-1] == 3:
        Ans = random.choice(MedQs)
        Full = Ans
        MedQs.remove(Full)
    else:
        Ans = random.choice(EasyQs)
        Full = Ans
        EasyQs.remove(Full)

elif (DiffQs[-1] == 3) or (DiffQs[-1] == 2) or (EasyQs == [] and MedQs == []):
    Ans = random.choice(HardQs)
    Full = Ans
    HardQs.remove(Full)
    print("Hard")
    print (HardQs)
elif (DiffQs[-1] == 1) or (HardQs == []) or (EasyQs == []):
    Ans = random.choice(MedQs)
    Full = Ans
    MedQs.remove(Full)
    print("Medium")
    print (MedQs)
elif (HardQs == [] and MedQs == []):
    Ans = random.choice(EasyQs)
    Full = Ans
    EasyQs.remove(Full)
    print("Easy")
    print (EasyQs)
```

```
def splitQs():
    global EasyQs, MedQs, HardQs
    count = 0
    for count in range(0, len(aClue)):
        AnsQuestion = aClue[count]
        if int(AnsQuestion[2]) == 3:
            HardQs.append(AnsQuestion)
        elif int(AnsQuestion[2]) == 2:
            MedQs.append(AnsQuestion)
        elif int(AnsQuestion[2]) == 1:
            EasyQs.append(AnsQuestion)
        count = count+1
    return
```

Function to create the difficulty arrays. (arrays of questions within each difficulty)

This code randomly chooses a clue from within each difficulty level array

I will then match this to the expected answer using the if statement I made above.

Currently on the first clue, the clue is not being deleted from the necessary array and that is why "Ahhhhh its not working" is being output.

36.43. Problem solution

I realised that the program was comparing the integer against the string of Full[2]. To correct this I cast Full[2] to an integer.

```
int(Full[2]) == 3:
```

In the process of fixing this issue I split the larger if statement into 2 smaller external subprograms.

```
def ClueAns(QuestionFull, Answer):
    for event in pygame.event.get():
        # Did the user hit a key?
        if event.type == KEYDOWN:
            #place holder to stop a build up of key presses
            pass
    if str(QuestionFull.upper()) == Answer.upper():
        #print statements to test
        print("True")
        print(QuestionFull.upper())
        print(Answer.upper())
        return True
    elif str(QuestionFull.upper()) != Answer.upper():
        #print statements to test
        print(QuestionFull.upper())
        print(Answer.upper())
        print("False")
        return False
    else:
        print("oh no something went wrong")
        return
```

Function ClueAns() to check question answers and output weather it is correct or incorrect.

```
def pausedQmanage(LastQ):
    #question selection
    print(DiffQs[-1])
    print(LastQ[-1])
    if not LastQ[-1]: #boolean
        if LastQ[-1] == 3:
            Ans = random.choice(MedQs)
            Full = Ans
            MedQs.remove(Full)
        else:
            Ans = random.choice(EasyQs)
            Full = Ans
            EasyQs.remove(Full)

    elif (DiffQs[-1] == 3) or (DiffQs[-1] == 2) or (EasyQs == [] and MedQs==[]) or
    (MedQs == []):
        Ans = random.choice(HardQs)
        Full = Ans
        HardQs.remove(Full)
        print("Hard")
        print (HardQs)
    elif (DiffQs[-1] == 1) or (HardQs == []) or (EasyQs == []):
        Ans = random.choice(MedQs)
```

Function pausedQmanage(LastQ) to select questions to be answered by the user specifically not the first one.

```

    Full = Ans
    MedQs.remove(Full)
    print("Medium")
    print (MedQs)
elif (HardQs == [] and MedQs == []):
    Ans = random.choice(EasyQs)
    Full = Ans
    EasyQs.remove(Full)
    print("Easy")
    print (EasyQs)
else:
    print(failed)
    Full = "Failed"
return Full

```

```

def Qmanage():
    #declared in initialise
    global Num, EasyQs, MedQs, HardQs, LastQ
    if Num == 0:
        Ans = random.choice(aClue)
        Full = Ans
        print(int(Full[2]))
        if int(Full[2]) == 3:    #define first Q
            HardQs.remove(Full)
            print("removed hard")
        elif int(Full[2]) == 2:
            MedQs.remove(Full)
            print("removed medium")
        elif int(Full[2]) == 1:
            EasyQs.remove(Full)
            print("removed easy")
        else:
            print("Ahhhhh its not working")
            Num = Num + 1
            LastQ = [Ans[2]]
    else:
        Full = pausedQmanage(LastQ)

# the first set question is not being removed from the array.
return Full

```

Function Qmanage() to select the first question to be given and then call pausedQmanage() to select questions after the first one.

```

elif paused:
    #paused
    #print(aClue)
    if Lives == 0:
        GameOver = True
    if not GameOver:

        print(EasyQs)
        print(MedQs)
        print(HardQs)

        QuestionFull = Qmanage()

```

Paused loop to implement the score and ensure that game is over when there are no more posnClues left.

```

print(QuestionFull)
print(QuestionFull[0])
Answer = str(input("> "))
if (ClueAns(QuestionFull[1], Answer) is True):
    Question(QuestionFull)
    Score = Score + 1
    print(Score)
    print(posnClue)
    if not posnClue: # If posnClue is empty
        print("All clues solved! Ending the game.")
        Comp = True
        GameOver = True
    else:
        LastQ.append(True)
        print(LastQ)
        paused = False # Resume the game

#PlayGame(aMazeRecord, aClueFile, aUserRecord, HeroAvatarFile)
elif (ClueAns(QuestionFull[1], Answer) is False) and (Score >= 1):
    Score = Score - 1
    print(Score)
    print(posnClue)
    #game over if no more clues
    if posnClue == []:
        print(posnClue)
        GameOver = True
        Comp = True
    else:
        LastQ.append(False)
        paused = False #resume game

else:
    GameOver = True

```

I have added a completion variable to be saved to the database to ensure that I can sort values by whether the game was successfully completed or not.

```

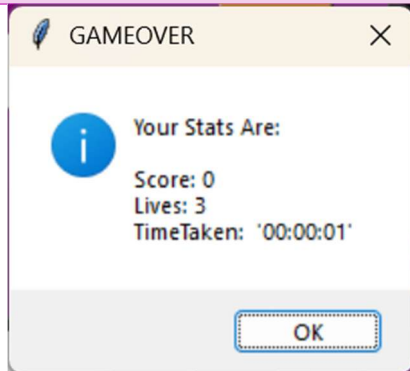
return[Score, Lives, TimeTaken, NumEasy, NumMed, NumHard, A, Comp]

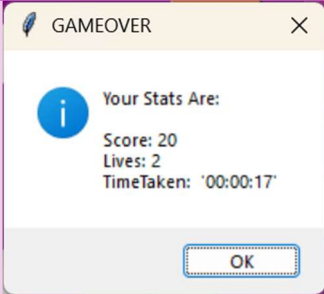
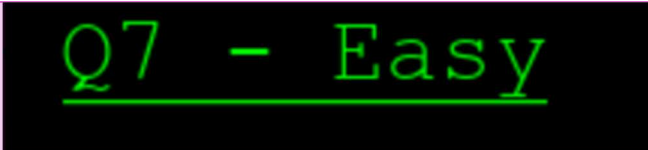
#link table:
User_Maze = '''CREATE TABLE IF NOT EXISTS User_Maze(
UMID integer primary key autoincrement,
Score integer,
TimeTaken time,
NumEasy integer,
NumMedium integer,
NumHard integer,
UserID integer NOT NULL,
MazeID integer NOT NULL,
COMP boolean Default 0,
FOREIGN KEY (UserID) REFERENCES User(UserID),
FOREIGN KEY (MazeID) REFERENCES Maze(MazeID)
)'''

```

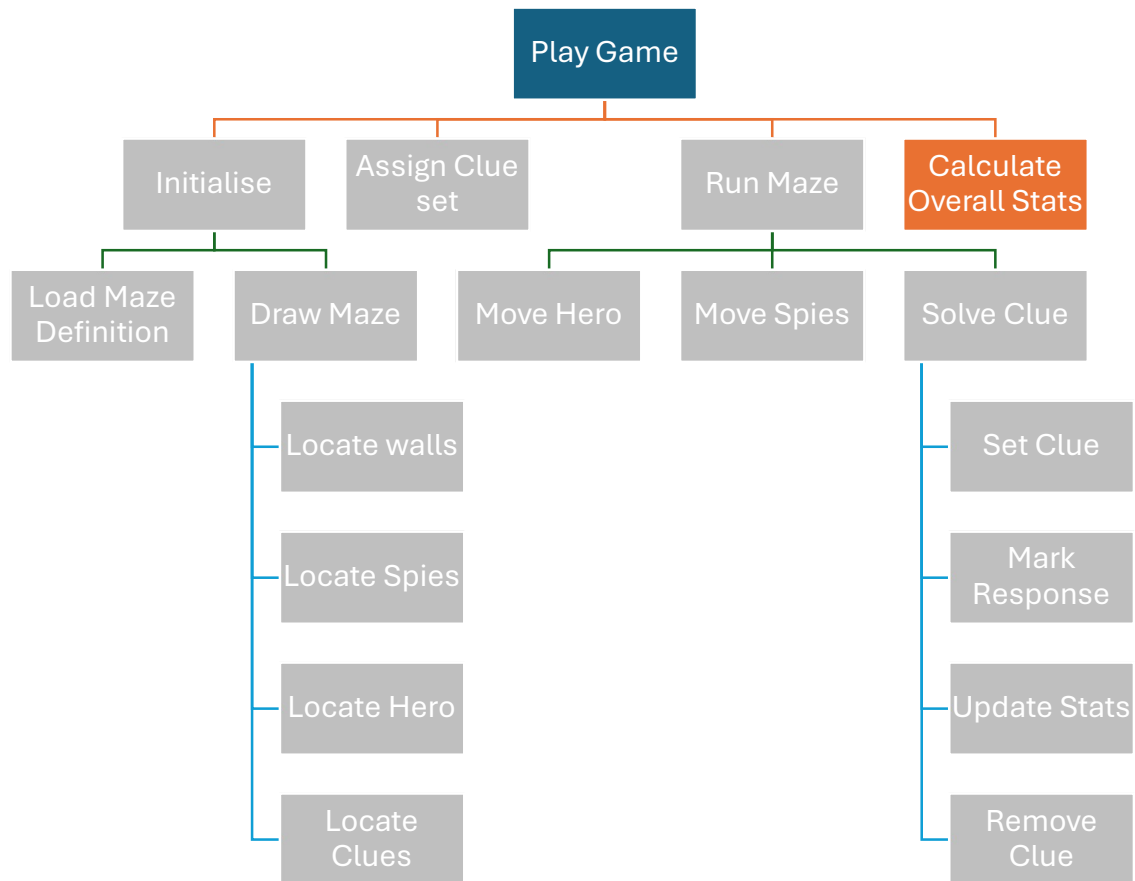
Adding a Boolean variable which is true if the game is successfully completed. (0 or 1)

36.44. Test Plan for Questions

Test No.	Description	Input Data	Expected Outcome	Actual Outcome	P/F
1	First Question is randomly selected of any difficulty	Hero Posn = CluePosn[0]	Any Difficulty	 (correctly answered)	P
2	If question before is correct and it is a Medium or Easy Clue, the difficulty will be increased one level on the next question. Or the difficulty level will stay the same if the first question is a hard clue.	Hero Posn = CluePosn[1] Correct answer to 1	Difficulty +1 If difficulty is hard remain	 (incorrectly answered)	P
3	If the question before is incorrect, the difficulty of this clue will decrease unless the previous clue was already an Easy clue.	Hero Posn = CluePosn[1] incorrect answer to 1	Difficulty -1 If difficulty is easy remain	 (correctly answered)	P
4	Q1 incorrect GameOver = true	Hero Posn = CluePosn[0] incorrect answer	quit		P

5	If CluePosn is empty, then GameOver and Completed is true	CluePosn = 0	Game Over		P
6	If Easy Questions are finished, then the next clues selected will be Medium Clues	Easy == 0	Medium question even if previous was wrong	This is quite difficult to achieve due to the likelihood of getting a negative score and triggering Game Over for getting too many clues wrong, before this condition is true. (my original presidency plan was to have 2 questions of the same level right before moving on but I developed it to move on after 1 question to make it more interesting)	F
7	If Hard Questions are finished, then the next clues selected will be Medium Clues	Hard == 0	Medium question even if previous Hard or Medium Clue was answered correctly		P
8	If Easy and Medium Questions are finished, then the next clue selected will be Hard	Easy == 0 Medium = 0	Hard question even if previous was wrong	This is quite difficult to achieve due to the likelihood of getting a negative score and triggering Game Over before this condition is true. (my original presidency plan was to have 2 questions of the same level right before moving on but I developed it to move on after 1 question to make it more interesting)	F
9	If Medium and Hard Questions are finished, then the next selected will be Easy	Medium = 0 Hard == 0	Easy question even if previous was wrong		P

36.45. I am now going to continue to consider the different elements of my decomposed system to develop the calculation algorithm for accuracy (one of the overall stats). This is shown by the coloured box on the diagram below:



36.46. Accuracy Calculation

At the end of the game, I would like to output Accuracy as a field to allow the teacher stakeholder to assess the level of competence of the students using the programme, even if they didn't complete the game.

For this I need to do two things:

1. Create a function to calculate accuracy.
2. Add an Accuracy field to the database for later saving.

I started with problem 1 above and used the reference

<https://uk.indeed.com/career-advice/career-development/weighted-average>²¹

to work out how to add weights to a calculation because I wanted the hard questions to carry more weighting than the easy questions.

Accuracy Calculation Algorithm

36.47. The following is my accuracy calculation algorithm:

```
//earlier in the program I need to have two arrays in which the question difficulty is always added to one and added to the other only if the answer is right.
```

```
Function Accuracy
```

```
    //To find the Total number of questions solved
```

```
    Iterate through the full list
```

```
        If value at array index = 1 then
```

```
            Total easy = +1
```

```
        Else if value at array index = 2 then
```

```
            Total Medium = +1
```

```
        Else if value at array index = 3 then
```

```
            Total Hard = +1
```

```
        Endif
```

```
    //To find the Total number of correct questions solved
```

```
    Iterate through the correct list //repeat iteration above for solved questions
```

```
    Accuracy Easy = Solved Easy / Total easy
```

```
    Accuracy Medium = Solved Medium / Total Medium
```

```
    Accuracy Hard = Solved Hard / Total Hard
```

```
    Weighted Easy = Easy Weight * Accuracy Easy
```

```
    Weighted Medium = Medium Weight * Accuracy Medium
```

```
    Weighted Hard = Hard Weight * Accuracy Hard
```

```
    Overall Weighted Accuracy = Weighted Easy + Weighted Medium + Weighted Hard
```

```
    Overall Weighted Accuracy = * 100           //to make it into a percentage
```

```
    Round to 0dp
```

```
    Return Accuracy
```

```
End Function
```

36.48. Coding the function

I will call the accuracy function near the end of the PlayGame function where it will be added to the database along with the other stats at Game Over. This is my coding for the accuracy function, which includes identifying and overcoming errors:

```
def Accuracy():
    LastQs.pop(0)      #to remove the status of the previous question
    print(LastQs)      #print test for LastQs
    length = len(LastQs) #to know how many time
    TotEasy = 0
    TotMed = 0
    TotHard = 0
    for questions in range(0,length):      #to find total numbers answered
        if LastQs[questions] == 1:
            TotEasy = TotEasy + 1      #to find the total number of Easy
questions asked
        elif LastQs[questions] == 2:
            TotMed = TotMed + 1
        elif LastQs[questions] == 3:
            TotHard = TotHard + 1
    print(TotEasy)      #test Totals
    print(TotMed)
    print(TotHard)

    print(DiffQs)      #Re-do for DiffQs(The correctly solved questions)
    length = len(DiffQs)
    global SolEasy, SolMed, SolHard
    SolEasy = 0
    SolMed = 0
    SolHard = 0
    for solved in range(0, length):      #to find numbers solved
        if DiffQs[solved] == 1:
            SolEasy = SolEasy + 1
        elif DiffQs[solved] == 2:
            SolMed = SolMed + 1
        elif DiffQs[solved] == 3:
            SolHard = SolHard + 1
    print(SolEasy)
    print(SolMed)
    print(SolHard)

    #calc Accuracy of each Lvl
    AccEasy = SolEasy/TotEasy
    AccMed = SolMed/TotMed
    AccHard = SolHard/TotHard

    #weighted accuracy for each accuracy
    WAccEasy = (AccEasy * EWeight)
    WAccMed = (AccMed * MWeight)
    WAccHard = (AccHard * HWeight)

    #calc overall accuracy
    OverallWA = WAccEasy + WAccMed + WAccHard      #final weighted average
    OverallWA = OverallWA * 100      #percentage
```

```
#rounding
OverallWA = round(OverallWA)

OverallAcc = str(OverallWA)+ "%"
return OverallAcc
```

```
Traceback (most recent call last):
  File "C:\Users\RosaB\Downloads\Cyber-Quiz-Maze\PlayGame01.py", line 740,
in module>
    PlayGame(aMazeRecord, aClueFile, aUserRecord, HeroAvatarFile)
  File "C:\Users\RosaB\Downloads\Cyber-Quiz-Maze\PlayGame01.py", line 267,
in ayGame
    A = Accuracy()
  File "C:\Users\RosaB\Downloads\Cyber-Quiz-Maze\PlayGame01.py", line 708,
in curacy
    AccEasy = SolEasy/TotEasy
ZeroDivisionError: division by zero
>>>
```

To Solve this I have decided to compare Sol(Difficulty) to 0 to ensure that if it is equal to 0 then WAcc(Difficulty) will automatically be 0. This will stop divisions like 0/0 and 0/4 which lead to maths errors if the User dies without solving any questions of a certain level.

```
if SolEasy != 0:
    AccEasy = SolEasy/TotEasy
    print(AccEasy)
    WAccEasy = (AccEasy * EWeight)
else:
    WAccEasy = 0
```

To Test this I purposely hit a Spy 3 times to ensure my Hero died without solving questions.
This led to another error

```
Traceback (most recent call last):
  File "C:\Users\RosaB\Downloads\Cyber-Quiz-Maze\PlayGame01.py", line 685
in module>
    PlayGame(aMazeRecord, aClueFile, aUserRecord, HeroAvatarFile)
  File "C:\Users\RosaB\Downloads\Cyber-Quiz-Maze\PlayGame01.py", line 267
in ayGame
    A = Accuracy()
  File "C:\Users\RosaB\Downloads\Cyber-Quiz-Maze\PlayGame01.py", line 604
in curacy
    LastQs.pop(0)      #to remove the status of the previous question
IndexError: pop from empty list
>>>
```

To fix this I wrapped an if statement around all the code in the accuracy calculation except the last two lines with the selection "if LastQs:" so that if there were no values in LastQs I could set the Accuracy to 0.

This led to a further error message:

```

Traceback (most recent call last):
  File "C:\Users\RosaB\Downloads\Cyber-Quiz-Maze\PlayGame01.py", line 688,
  in <module>
    PlayGame(aMazeRecord, aClueFile, aUserRecord, HeroAvatarFile)
  File "C:\Users\RosaB\Downloads\Cyber-Quiz-Maze\PlayGame01.py", line 269,
  in PlayGame
    NumEasy = SolEasy
NameError: name 'SolEasy' is not defined
>>>

```

To fix this I globally declared the variables before the if so they would still be able to be used in the Game Function.

```

0%
Game Stats
Score: 0          Lives: 0          TimeTaken: 00:04:20
>>>

```

The game runs correctly and the output is as expected

The final code for Accuracy:

```

def Accuracy():
    global SolEasy, SolMed, SolHard
    SolEasy = 0
    SolMed = 0
    SolHard = 0
    TotEasy = 0
    TotMed = 0
    TotHard = 0
    if LastQs:
        LastQs.pop(0)      #to remove the status of the previous question
        print(LastQs)      #print test for LastQs
        length = len(LastQs) #to know how many time

        for questions in range(0,length):      #to find total numbers
answered
            if LastQs[questions] == 1:
                TotEasy = TotEasy + 1      #to find the total number of Easy
questions asked
            elif LastQs[questions] == 2:
                TotMed = TotMed + 1
            elif LastQs[questions] == 3:
                TotHard = TotHard + 1
        print(TotEasy)      #test Totals
        print(TotMed)
        print(TotHard)

        print(DiffQs)      #Re-do for DiffQs(The correctly solved questions)
        length = len(DiffQs)
        for solved in range(0, length):      #to find numbers solved
            if DiffQs[solved] == 1:
                SolEasy = SolEasy + 1
            elif DiffQs[solved] == 2:
                SolMed = SolMed + 1
            elif DiffQs[solved] == 3:
                SolHard = SolHard + 1
        print(SolEasy)
        print(SolMed)
        print(SolHard)

```

```

#calc Accuracy of each lvl
#ref23 weighted Accuracy
#https://www.khanacademy.org/computing/pixar/environment-modeling-
2/mathematics-of-parabolas2-ver2/e/weighted-averages-on-a-line
if SolEasy != 0:
    AccEasy = SolEasy/TotEasy
    print(AccEasy)
    WAccEasy = (AccEasy * EWeight)
else:
    WAccEasy = 0

if SolMed != 0:
    AccMed = SolMed/TotMed
    print(AccMed)
    WAccMed = (AccMed * MWeight)
else:
    WAccMed = 0

if SolHard != 0:
    AccHard = SolHard/TotHard
    print(AccHard)
    WAccHard = (AccHard * HWeight)
else:
    WAccHard = 0

#calc overall accuracy
OverallWA = WAccEasy + WAccMed + WAccHard #final weighted
average
OverallWA = OverallWA * 100 #percentage
#rounding
OverallWA = round(OverallWA)
else:
    OverallWA = 0

OveralAcc = str(OverallWA)+ "%"
return OveralAcc

```

36.49. Adding an Accuracy field to the Database

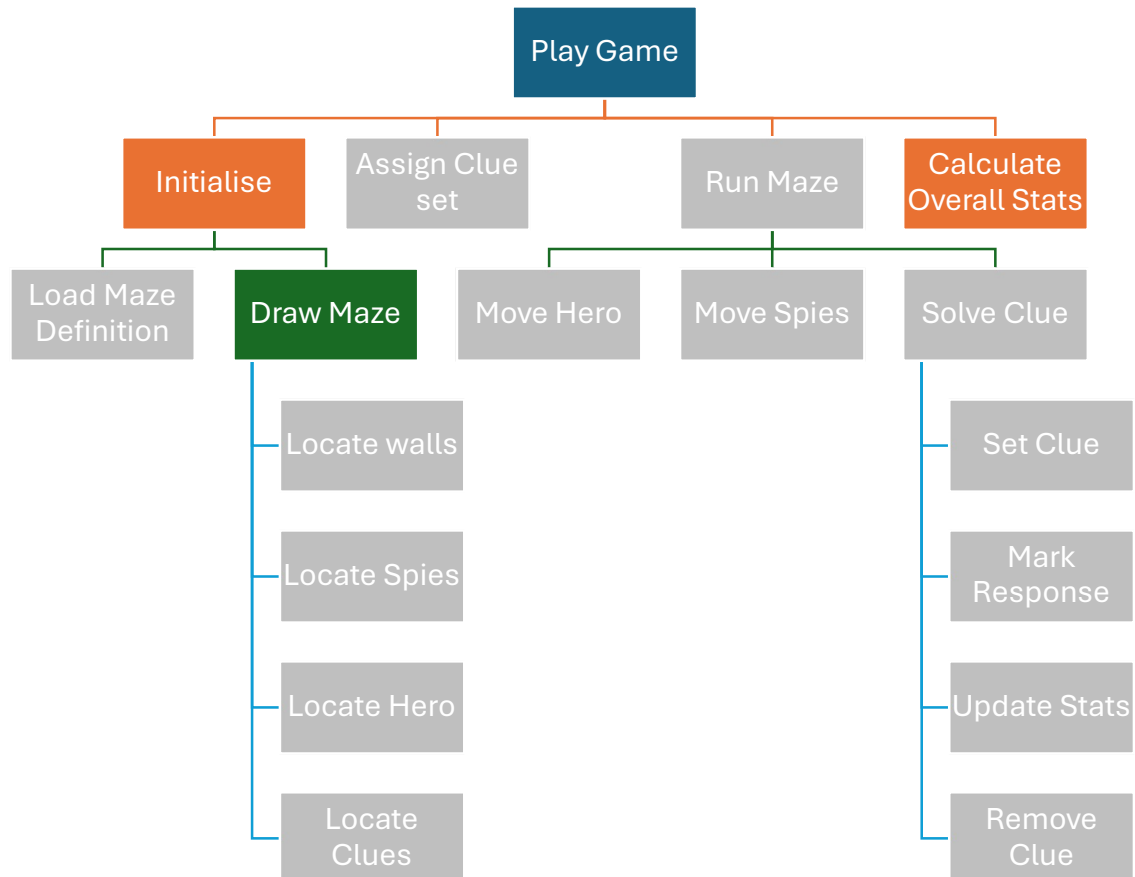
I added an accuracy field to the database to allow for it to be displayed to the teacher stakeholder when looking for information about the progress of their students. It allows for students not to complete the game but for it to record their progress anyway.

```
#link table:
User_Maze = '''CREATE TABLE IF NOT EXISTS User_Maze(
UMID integer primary key autoincrement,
Score integer,
TimeTaken time,
NumEasy integer,
NumMedium integer,
NumHard integer,
Accuracy text NOT NULL,
UserID integer NOT NULL,
MazeID integer NOT NULL,
COMP boolean Default 0,
FOREIGN KEY (UserID) REFERENCES User(UserID),
FOREIGN KEY (MazeID) REFERENCES Maze(MazeID)
)'''
```



Accuracy Field added to the Maze table to make sure that when the data is added to the User data table after game completion

36.50. I am now going to continue to consider the different elements of my decomposed system to develop on screen stats and text that will be seen during the game play. The coloured boxes below are the areas where I believe I will need to adapt for on screen stats:



36.51. Adding Stats to the game screen

I then moved on to work on the GUIs for this project, which is recorded in the 3rd Iteration. I did this to give me an idea of how GUIs work and to have a break from the game development because I was running out of ideas of how to proceed with stats on the screen, spy movement and the timer. I was hoping that transferring onto the GUI side of the project would give me fresh ideas to work with when I returned to this subject.

I am now returning to the Stats display after working on the GUIs.

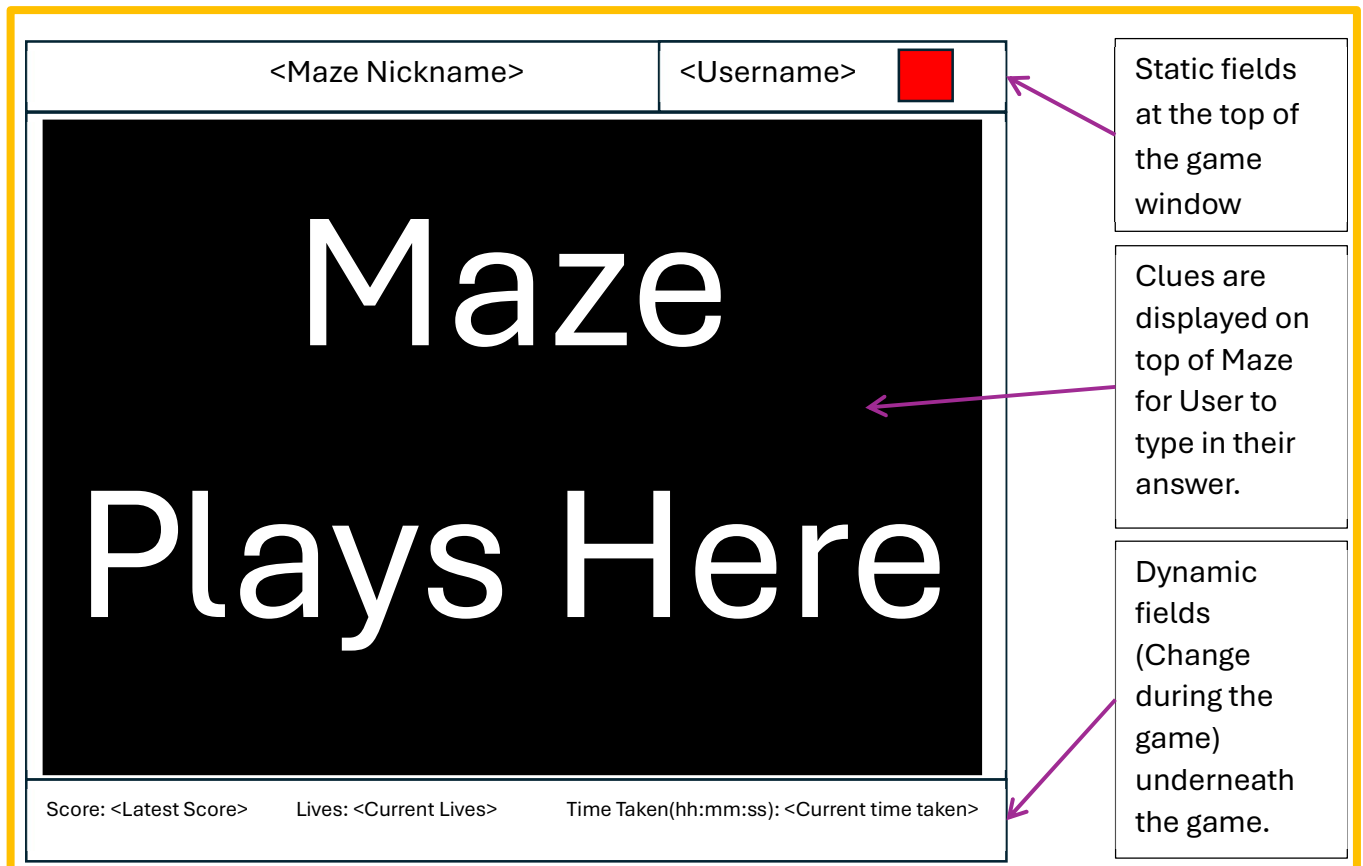
Status of My Current Game:

- I can draw and update the maze
- I can move the hero correctly

My next challenge is to add the game stats view. I would like to display and update the following whilst a User is playing the maze game:

- Score so far - dynamic
- Time Taken so far - dynamic
- Lives - dynamic
- Maze nickname - static
- Avatar + Username – Static

This is my initial design idea for how the game stats visuals might look:



I used the reference <https://stackoverflow.com/questions/10077644/how-to-display-text-with-font-and-color-using-pygame>²⁴ to learn how to display text on screen in python pygame and I implemented my learning as shown below:

```

else:
    CellColour = colEmpty

    pygame.draw.rect(SCREEN, CellColour, rect, 0)

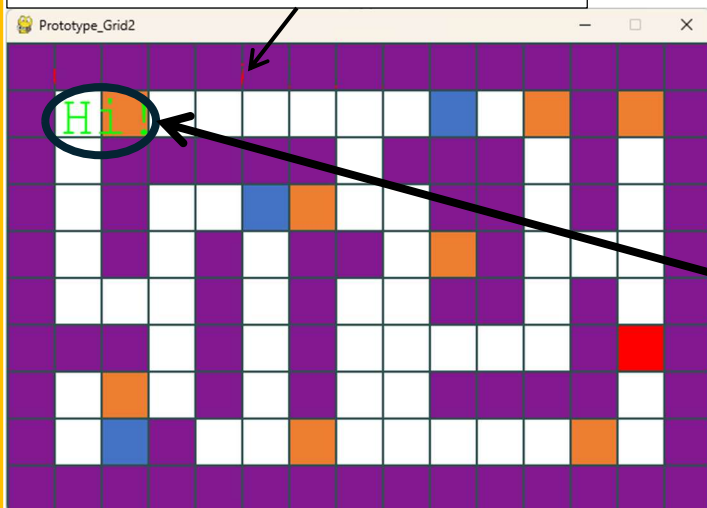
my_ft_font = pygame.freetype.SysFont('Courier New', 50)
my_ft_font.render_to(SCREEN, (50, 50), ("Hi!"), (0, 255, 0))
pygame.display.update()
return

```

If you display text in initialise it gets overwritten every time the surface is refreshed

In DrawMaze() ,

1. Draw the updated maze
2. Draw the text on top

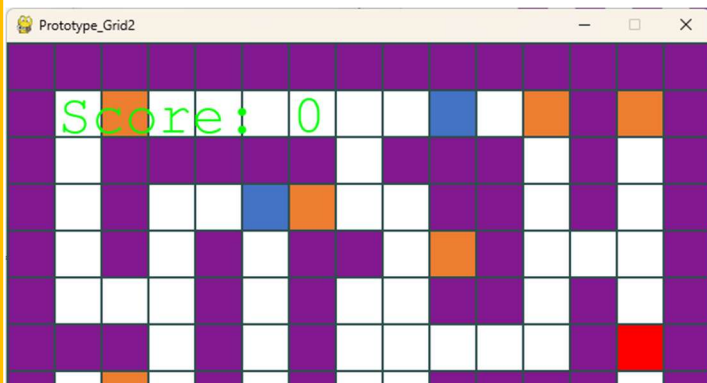


Evaluation

The text must be drawn after the maze if it is to appear on top.

I can set the font style, colour, size and position on the surface

Question: Do I need one surface for the text and another for the maze?



I have added a score variable on screen.

From here I need to create 2 different boxes of the same height as 1 row. I will place them at the bottom of the screen and the top.

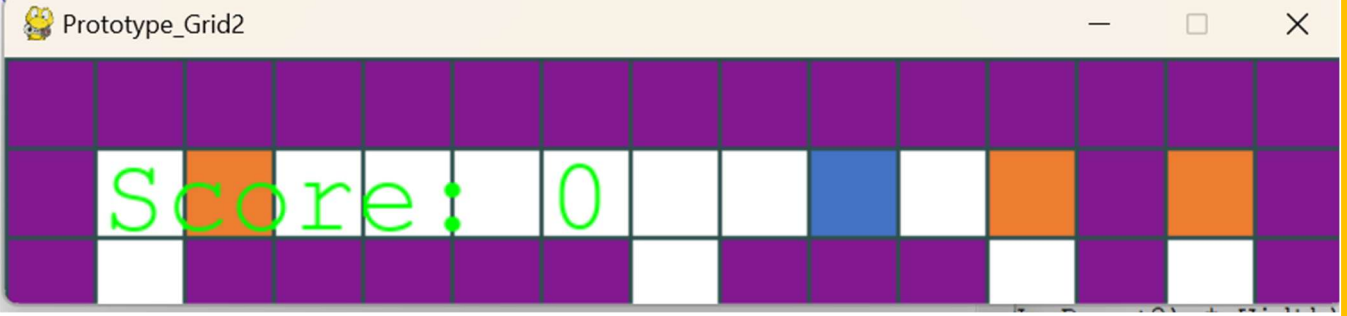
```

#prototype - add texts fields to dieplay data
#ref:https://stackoverflow.com/questions/10077644/how-to-display-text-with-
my_ft_font = pygame.freetype.SysFont('Courier New', 50)
my_ft_font.render_to(SCREEN, (50, 50), ("Score: "+str(Score)), (0, 255, 0))

```

36.52. Add bars to contain the info at the top and bottom

This resulted in an Error that needed to be investigated and resolved as follows:



```
#Draw the maze as a grid
#for the background
BackWidth = (NumCols * Width) + ((NumCols+1) * Thick)
BackHeight = (NumRows+2 * Width) + ((NumRows+3) * Thick)
BackColour = pygame.Color(colBG)
#ref: https://www.pygame.org/docs/ref/color\_list.html17
```

This calculation requires brackets around it otherwise according to bidmas the 2 will be times by width first then added to the Number of Rows.

Below is the corrected version that removes the logic error

```
#Draw the maze as a grid
#for the background
BackWidth = (NumCols * Width) + ((NumCols+1) * Thick)
BackHeight = ((NumRows+2) * Width) + ((NumRows+3) * Thick)
BackColour = pygame.Color(colBG)
#ref: https://www.pygame.org/docs/ref/color\_list.html17

#Set up a pygame window
pygame.init()
pygame.display.set_caption("Prototype_Grid2")
```

I edited the calculations for the height of the total screen to allow for space for stats to be added later after it is shuffled equally top and bottom.

At the moment, the extra space is added at the bottom.



I will now edit the code to ensure the blank space is evenly distributed at the top and bottom allowing the maze to sit nicely in the middle of the screen.

```
def DrawMaze():  
    #draw the grid of cells  
    global SCREEN #because procedure m  
    for Row in range(2, NumRows+2):  
        Y = (Row)*Thick + (Row-1)*Width
```

I changed the range of the for loop and added 1 to both of the range values.

This changed the calculations of where to place the blocks.

This caused a logic error where the rows are offset upwards one and the bottom row is empty.

I fixed this as follows:

```
#need to select cell colour according to precedence  
cellCoord = (Row-1, Col-1) #tuple for cell coordinate  
#selection and find coordinate  
  
cellCoord = (Row-2, Col-1)
```

To sort this, I need to change the cellCoord of the row to change the relation to the posn arrays.

36.53. Adding the text on top of the box

Because this text is always the same on screen, I have decided to put it in the initialise procedure.

```
#prototype - add texts fields to dieplay data
#ref:https://stackoverflow.com/questions/10077644/how-to-display-text-with-
my_ft_font = pygame.freetype.SysFont('Courier New', 30)
#title
my_ft_font.render_to(SCREEN, (50, 13), (Nickname), (0, 255, 0))
#username
my_ft_font.render_to(SCREEN, (390, 13), (Username), (0, 255, 0))
```

I used the method from above to add text above I just changed some of the styling to fit into the bar at the top.



I need to know how to add a scaled image on to the screen and I used this reference to help me learn how to do this: <https://gamedevacademy.org/pygame-scale-image-tutorial-complete-guide/>²⁵

```
#add a small avatar icon
#ref: https://gamedevacademy.org/pygame-scale-image-tutorial-complete-guide
Avatar = pygame.image.load(AvatarFile) #load in the avatar file
scaled_Avatar = pygame.transform.scale(Avatar, ((Width-10), (Width-10))) #
SCREEN.blit(scaled Avatar, (375 - Width, 7))
```

Steps:

1. Load the image file into the pygame file
2. Scale the image to the correct size.
3. Display the image at a specific position



36.54. Adding the text below the box

For the constantly changing text under the game, it will need to be updated and drawn in the draw maze procedure so that the score, lives and time will be updated alongside the game.

```
#below game box
Y = (NumRows+2)*Thick + (NumCols+1)*Width
X = (1)*Thick
ExWidth = (Width*(NumCols))+(Thick*(NumCols-1))
rect = pygame.Rect(X, Y, (ExWidth), Width)
CellColour = "black"
pygame.draw.rect(SCREEN, CellColour, rect, 0)

#prototype - add texts fields to dieplay data
#ref:https://stackoverflow.com/questions/10077644/how-to-display-text-with-24
my_ft_font = pygame.freetype.SysFont('Courier New', 30)
my_ft_font.render_to(SCREEN, (10, Y+10), ("Score:"+str(Score)), (0, 255, 0)
my_ft_font.render_to(SCREEN, ((ExWidth/4), Y+10), ("Lives:"+str(Lives)), (0, 255, 0)
my_ft_font.render_to(SCREEN, ((ExWidth/4)*2, Y+10), ("Time:"+str("'00:00:00'")), (0, 255, 0)

pygame.display.update()
return
```

At first the rectangle to house the values is drawn and then the values as strings. The values must be cast as integers cannot be entered.



36.55. Evaluation

I can draw static text (which will not change during the game) above the maze, and dynamic text (Score, lives and time) below the maze.

I have the following success criteria for dynamic text:

1. When the game starts
 - a. Score = 0
 - b. Lives = 3 //fixed (Maintenance: with further time, I would want to add the lives available to the maze record giving the Admin an opportunity to edit this.)
 - c. – can be varied according to the difficulty level / set according to the level of difficulty.
 - d. Timer = 00:00 (mm:ss) and counts up to max time allowed for this game.
2. The dynamic text must be updated when
 - a. A clue is solved correctly
 - b. A life is lost – (collision between spy and hero)
 - c. When another second has elapsed since the last one
3. The game ends when
 - a. All clue positions have been asked and removed. (clues asked)
 - b. All lives have been lost
 - c. The MaxTime for the game has elapsed

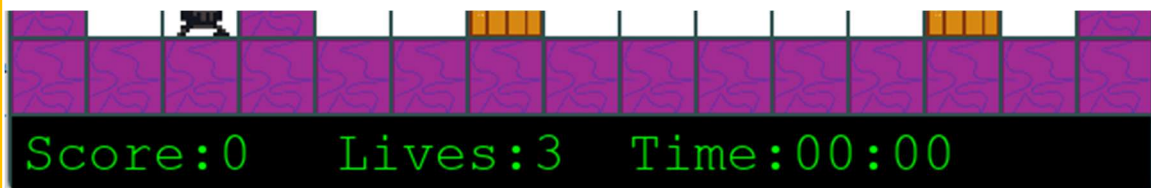
36.56. Development of the Dynamic text

```
#for displaying dynamic text|
global Lives, Score, MaxTime, ElapsedTime
Lives = 3
Score = 0
MaxTime = "00:00"
ElapsedTime = "00:00"
```

Start by making global variables in initialise which can be used for dynamic text

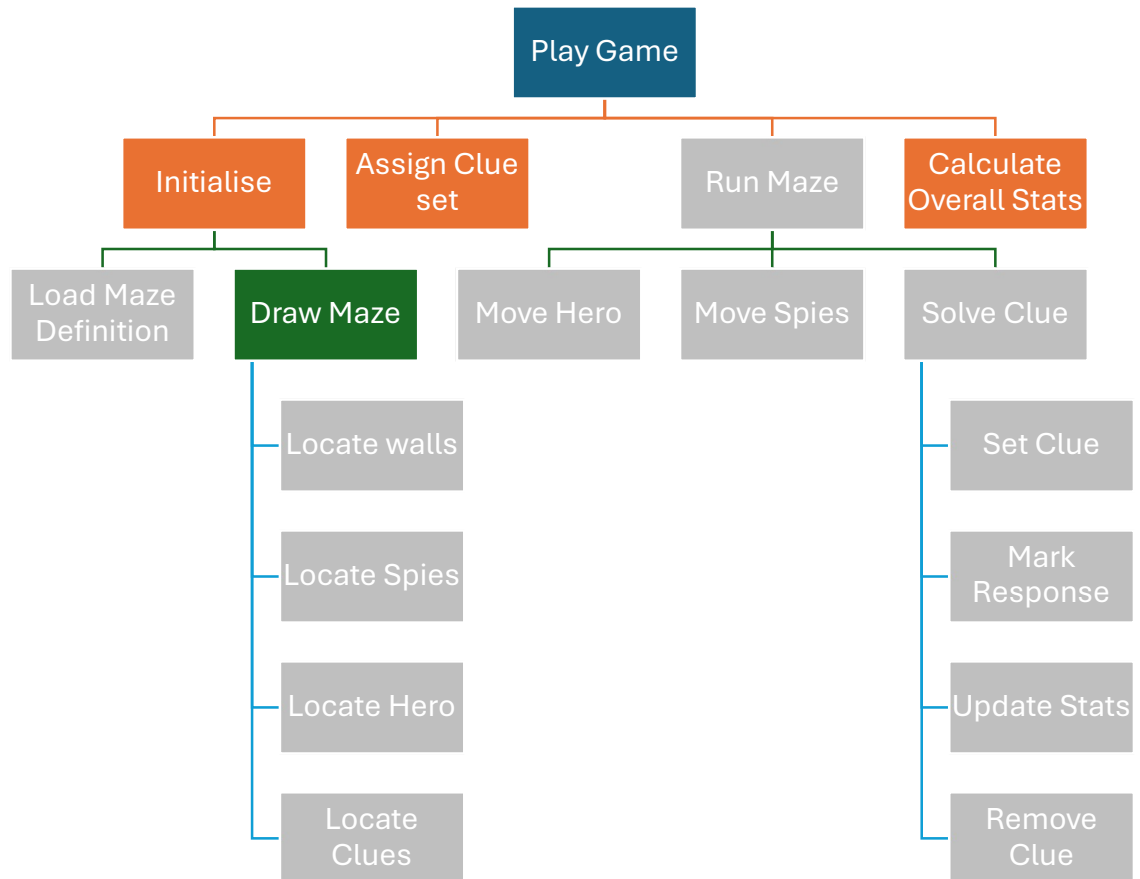
```
#prototype - add texts fields to display data
#ref:https://stackoverflow.com/questions/10077644/how-to-display-text-with-font-and-color-using-pygame 24
my_ft_font = pygame.freetype.SysFont('Courier New', 30)
my_ft_font.render_to(SCREEN, (10, Y+10), ("Score:"+str(Score)), (0, 255, 0))
my_ft_font.render_to(SCREEN, ((ExWidth/4 + 25), Y+10), ("Lives:"+str(Lives)), (0, 255, 0))
my_ft_font.render_to(SCREEN, ((ExWidth/4)*2 + 30, Y+10), ("Time:"+str(ElapsedTime)), (0, 255, 0))
```

In DrawMaze() whenever the maze is refreshed so is the dynamic text



The dynamic text is displayed correctly under the maze

36.57. I am now going to continue to consider the different elements of my decomposed system to develop an on-screen timer for my prototype and manipulate time to be able to pause time when necessary. This area of focus is shown by the coloured boxes on the diagram below:



36.58. Timer Development

```
import time
```

To calculate time I have imported the time module into PlayGame01

```
# Game timer setup
start_time = time.time() # Record the start time now
nowtime = start_time #used to update time elapsed

#question_timer = None # To handle individual question timers

while not GameOver:
    if not paused:
```

Start
measuring
time as the
game loop
starts

In my game loop I have added a way of updating elapsed time before calling DrawMaze().

It is working as shown on the right, but it is not updating the display time

```
CurrentTime = time.time() #right now
if int(CurrentTime-nowtime) >= 1:

    ElapsedTime = int(CurrentTime - start_time) # Elapsed
    nowtime = CurrentTime
    print(ElapsedTime)
```

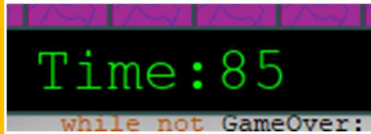
```
pygame 2.6.1 (SDL 2.28.4, Python 3.8.10)
Hello from the pygame community. https://
00:10:00
1.5
[(6, 13), (6, 13), (6, 10)]
1
2
3
4
5
6
7
8
9
10
11
[(6, 13), (6, 13), (6, 10)]
AFTER: row: 6 col: 13
```

Time:00:00

36.59. Updating Time

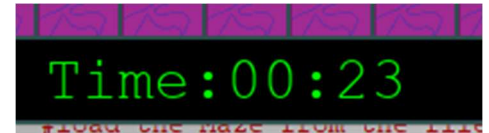
```
global Lives, Score, TimeTaken, ElapsedTime
```

ElapsedTime must be global in PlayGame in order to be able to edit it.



Currently the output is not formatted

The Time is now correctly formatted by using a subprogram and displaying TimePassed.



```
def FormatTime(secs):
    #function to format the seconds into mm:ss
    mins = secs // 60
    mm = str(mins)
    if mins < 10:
        mm = "0%s" %mm
    sec = secs - 60*mins
    ss = str(sec)
    if sec < 10:
        ss = "0%s" %ss
    Overall = "%s:%s" %(mm,ss)
    return Overall
```

Correct format according to my design

In the playgame loop I have added a test for the end of the game. (MaxTime = Elapsed)
The Max time allowed is calculated at the start of play game from the Maze definition.

```
TimePassed = FormatTime(ElapsedTime)
print(TimePassed)

#is it game over?
if ElapsedTime >= MaxTime:
    GameOver = True
    #game time has run out so quit loop
```

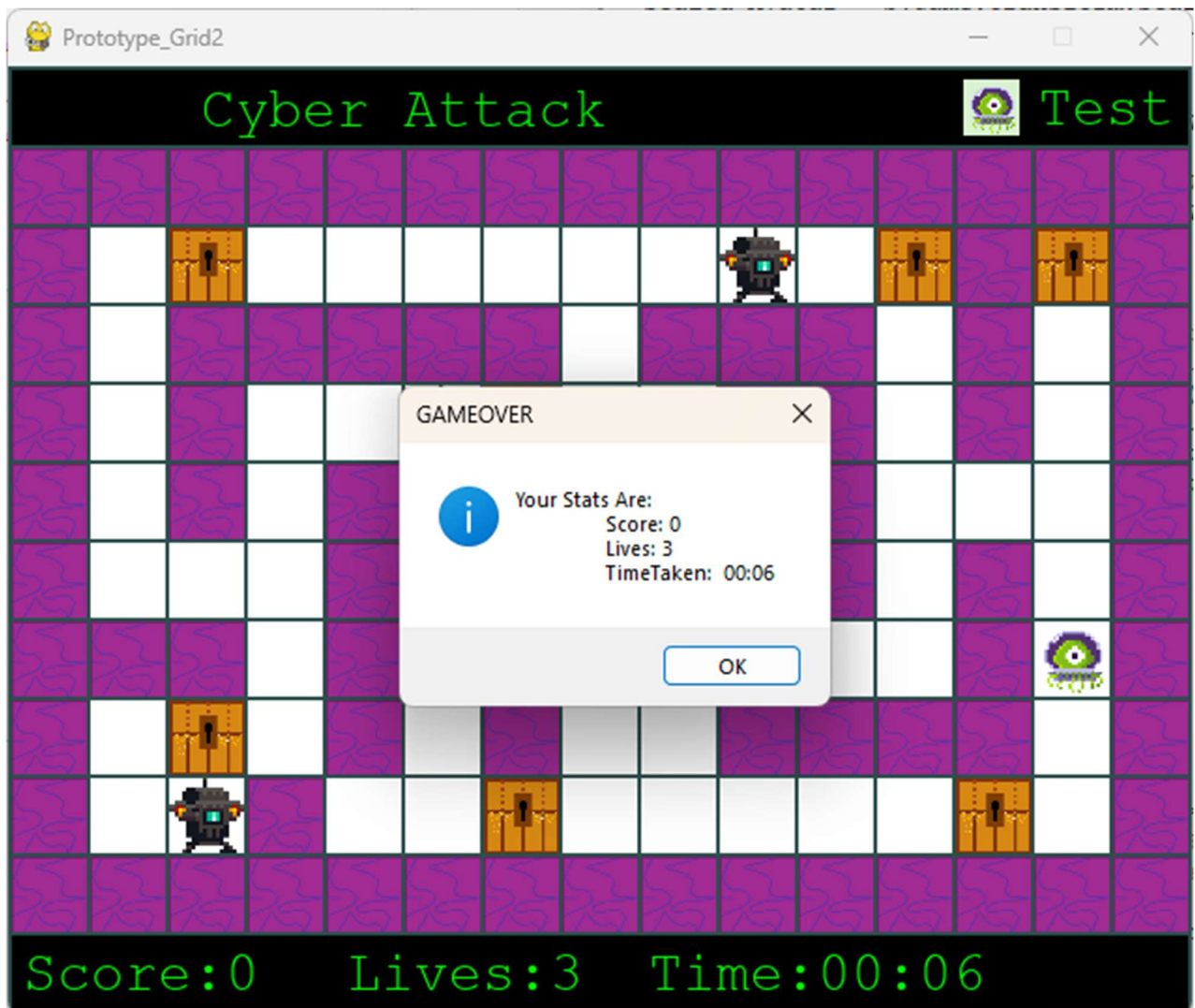
```
total_time = aMazeRecord[3]
print(total_time)
hours,mins,secs = total_time.split(":")
hrsecs = 60*60*int(hours)
minsecs = 60*int(mins)
secs = int(secs)
global MaxTime
MaxTime = hrsecs + minsecs + secs          #total game time in seconds
print(MaxTime)
```

After the game loop I have added a Game Over message box before the game is closed and the stats are saved to the database.

```
TimeTaken = TimePassed

#game over message
msg = "Your Stats Are:\n\tScore: %i\n\tLives: %i\n\tTimeTaken: %s"%(Score, Lives, TimeTaken)
mbox.showinfo("GAMEOVER", msg)

pygame.quit()    #close game
```



For a test max time of 00:06 the game correctly stops after a max time of 6 seconds and displays the game over message with stats.

36.60. **Evaluation**

My code now manages time including Game Over due to max time. I have also realised that when I play the game if I “pause” the game, the timer still runs behind the scenes and jumps once the game is unpaused.

36.61. **Correction**

To reset the time correctly, I created a paused time variable which will start running as soon as the game is paused. I also added a pause timer which consisted of a subprogram made to reset the timer to the value of the timer before the pause. This subprogram is shown below:

```
def PausedTime():
    global pause_start, start_time, nowtime
    #procedure to restart the timer
    pause_duration = time.time() - pause_start # How long the timer was paused
    start_time = start_time + pause_duration # Adjust the start time
    nowtime = time.time() # Reset nowtime to prevent immediate updates
    return
```

This subprogram does as described and finds the elapse of the Pause time and deletes it from the start time. This subprogram is called whenever the variable paused is changed to false (and game is not over).

```
if AnsCheck:
    print(Score)
    print(posnClue)
    if not posnClue: # If posnClue is empty
        print("All clues solved! Ending the game.")
        DrawMaze()
        Comp = True
        GameOver = True
        paused = False
        break #return to game over
    else:
        print(LastQs)
        PausedTime()
        paused = False # Resume the game

#PlayGame(aMazeRecord, aClueFile, aUserRecord, ...)
elif ((not AnsCheck) and (Score >= 1)):
    Score = Score - 1
    print(Score)
    print(posnClue)
    #game over if no more clues
    if posnClue == []:
        print(posnClue)
        Comp = True
        DrawMaze()
        GameOver = True
        paused = False
        break #return to game over
    else:
        print(LastQs)
        PausedTime()
        paused = False #resume game
else:
    GameOver = True
    paused = False
    break #return to game over
```

```
# Game timer setup
global start_time, nowtime, pause_start
start_time = time.time() # Record the start time now
nowtime = start_time # used to update time elapsed
pause_start = 0 # Time when the pause started
```

I have global'd the key variables for use above.

I have also added a pause_start variable so that the pause time is already declared

```
pause_start = time.time() # Note when the pause started
```

At the start of the while paused I have added this to note the time when the game has been paused.

After coding this I noticed that it would not be too difficult to code a timer for the Questions.

36.62. Code for Question timer (using aspects of the original timer)

I started by adding a globally declared nowP variable containing pause_start so that I am able to only display the whole numbers when they change.

To find the max question time, I took it from the Clueset file which contains a time allowance for each question. I then changed this formatted time into seconds.

```
QuestionFull = Qmanage()
# Display the question timer
global question_time_limit
question_time_limit = str(QuestionFull[3])
print(question_time_limit)
hours,mins,secs = question_time_limit.split(":")
hrsecs = 60*60*int(hours)
minsecs = 60*int(mins)
secs = int(secs)
question_time_limit = hrsecs + minsecs + secs #total game time in seconds
print(question_time_limit)
```

For the actual display and countdown I calculated these within the Questionpopup function in order to set conditions for the end of the timer.
(Automatic entry of whatever has been input even if NULL)

I started by displaying the max time so that I could change it later.

```
countdown = FormatQTime(question_time_limit)           #max question time

#time
my_ft_font.render_to(SCREEN, ((X+wid)-175, Y+10), ("%s" %countdown), (0, 255, 0))
```

These are set for the countdown to be changeable within the while not done.

```
while not done:
    current_question_time = time.time()
    global nowP, pause_elapse
    if int(current_question_time-nowP) >= 1:
        nowP = current_question_time      #reduces number of outputs
        pause_elapse = int(current_question_time - pause_start) # Time in the current pause
        countdown = (question_time_limit - pause_elapse)      #countdown
        countdown = FormatQTime(countdown)      #formatting
        print(countdown)      #test print
        if question_time_limit == pause_elapse:
            return user_text      #return whatever entered

    for event in pygame.event.get():      #check for events(key presses)
```

This code is at the start of the while not done and is made up of aspects of the original timer:

- The current time is taken every iteration which keeps the calculations accurate
- The number of output values is decreased by the first if statement.
- Time elapsed is calculated (though this time it is using the paused values)

The areas different to the original timer are:

- The global variables which allow the variables to be edited globally
- The countdown using the question_time_limit
- The if statement to compare the max to the current time paused.

36.63. To display the timer – developed post Question GUI

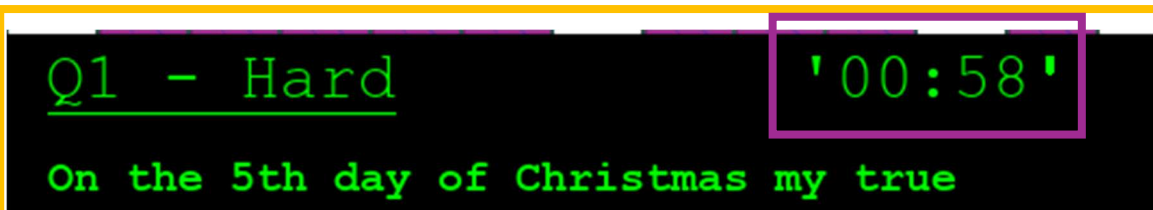
I wanted to have a question GUI to display my questions and subsequently my question timer. To display my timer, I had to develop my question GUI first and then come back to this part of the timer. I have placed the information about this part of the timer in the project at this point because I started developing the question timer before the question GUI and thought it would be confusing to split the timer development story.

```
pygame.draw.rect(SCREEN, green, text1_rect, 2) #redraw button

#draw a rectangle to be able to change the time
rect = pygame.Rect((X+wid)-175, Y+10, 100, (32))
CellColour = "black"
pygame.draw.rect(SCREEN, CellColour, rect, 0) #timer background
#time
my_ft_font.render_to(SCREEN, ((X+wid)-175, Y+10), ("%s" %countdown), (0, 255, 0))

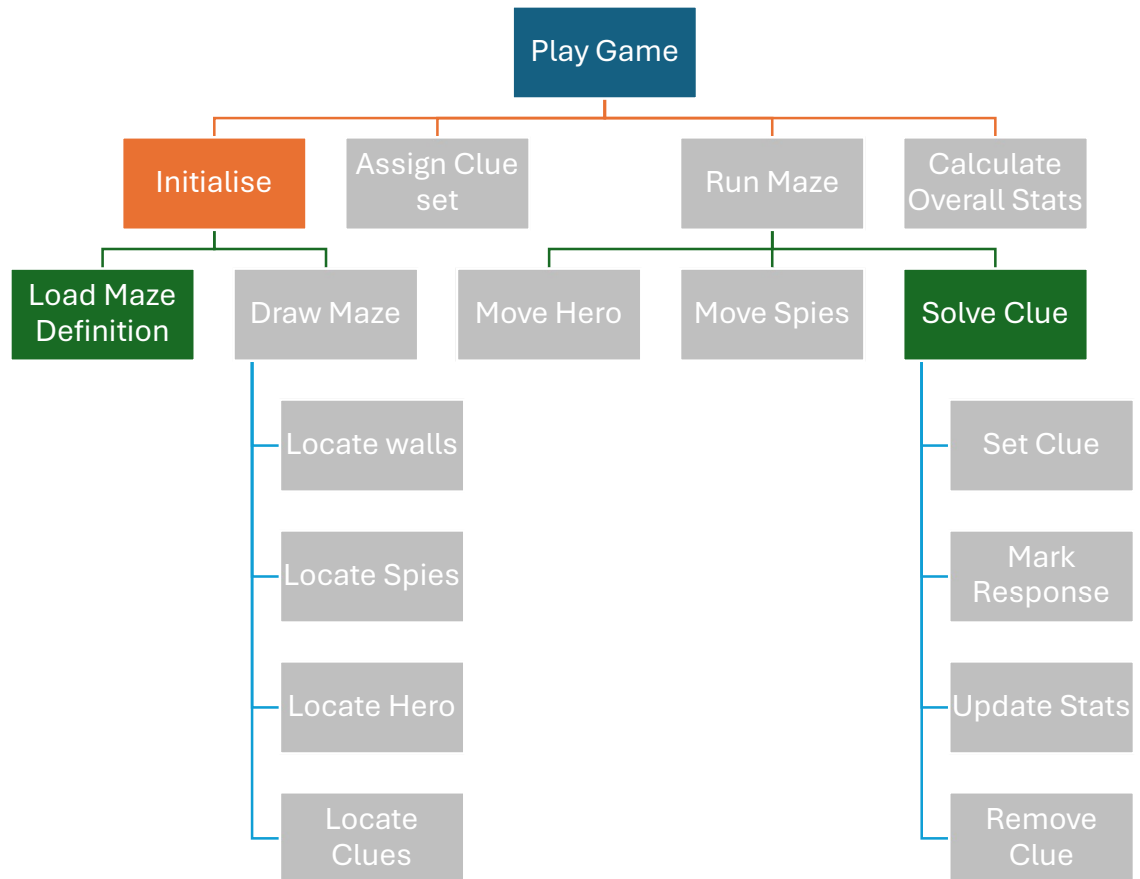
pygame.display.update()
```

To update the timer, the display needs to be updated so to do this I have reset the values including a black timer background to hide previous timer updates.



I have used a similar theme to the rest of the mini question GUI (developed later in this section) for the timer to fit in. I have also placed the timer in line with the Question number to look much more organised.

36.64. I am now going to continue to consider the different elements of my decomposed system implementing the maintenance point that I mentioned earlier about adding character icons. I will also develop the question GUI. These areas of focus are shown by the coloured boxes on the diagram below:



36.65. Adding images instead of colours to elements (Wall, Clue, Spy ...)

I have created a quick design for a wall which can be changed at a later date if the stakeholders don't like it.



The design uses a similar colour scheme to my current maze.

Using this reference as a guide, <https://gamedevacademy.org/pygame-scale-image-tutorial-complete-guide/>²⁵ I have learned how to input the wall icon on the left into my maze.

I used the same method as I used to add the avatar icon to the top bar of my game stats (see above). I loaded the image in a global variable in the initialise procedure and scaled the image where the grid is written (DrawMaze). This is shown below:

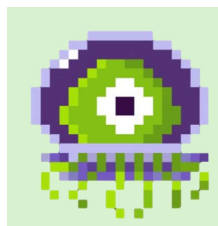
```
global Wall, Avatar, Clue, Spy
#ref: https://gamedevacademy.org/pygame-scale-image-tutorial-complete-guide/25
Wall = pygame.image.load("Wall.png") #load in the wall file
if cellCoord in posnWall:
    #CellColour = colWall
    #pygame.draw.rect(SCREEN, CellColour, rect, 0)
    scaled_Wall = pygame.transform.scale(Wall, ((Width), (Width))) #sc
    SCREEN.blit(scaled_Wall, rect.topleft)
```

I will do this for my other aspects of the game when I find images to use!



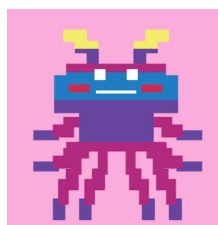
These are the other icons I added in the same way. I found them all online at the references below with the exception of the icon top left, which I made on PowerPoint. I found the other images at these references:

<https://www.deviantart.com/keatonmsk/art/Enemy-Pixel-Art-974179399>²⁶



https://www.freepik.com/premium-vector/pixel-art-illustration-artwork-design-character-bit-icon-symbol-insect-alien-monster-video-game_49213851.htm²⁷

<https://www.istockphoto.com/vector/space-game-pixel-art-aliens-and-spaceship-icons-gm1263111190-369681982>²⁸



<https://forums.tigsource.com/index.php?topic=46459.0>²⁹

I have loaded all of these apart from the chest without a background so that it looks as if it were moving around the maze. (Background is white)

I also decided to make the text of the stats green and in Courier new to make it similar to the retro games of the 70s as my Stakeholders suggested in an earlier survey.

36.66. Menu

To experiment with how to create buttons in pygame work, I decided to create a really simple menu GUI with buttons that I could develop into a mini question GUI. I started with a reference that I found to a helpful guide for creating text buttons in pygame, which I drew on to adapt it to suit my solution:

https://www.tutorialspoint.com/pygame/pygame_use_text_buttons.htm³⁰.

```
def MenuG():

    done = False
    Y = ((NumRows-1)*Thick + (NumRows-3)*Width)/2
    X = (Width)
    ExWidth = (Width*(NumCols))+(Thick*(NumCols-1))
    rect = pygame.Rect(X, Y, ((Width*14)-(Thick*2)), (Width*5))
    CellColour = "black"
    pygame.draw.rect(SCREEN, CellColour, rect, 0)

    #title of mini menu gui.
    my_ft_font = pygame.freetype.SysFont('Courier New', 60)
    my_ft_font.underline = True
    my_ft_font.render_to(SCREEN, (X+20, Y+10), ("Menu"), (0, 255, 0))

    green = (0, 255, 0)
    # Button text font
    button_font = pygame.freetype.SysFont('Courier New', 30)
    button_font.underline = False

    # Render text buttons
    text1_surface, text1_rect = button_font.render(" Restart ", green)
    text2_surface, text2_rect = button_font.render(" Resume  ", green)
    text3_surface, text3_rect = button_font.render(" Quit   ", green)

    # Set text into rectangles
    text1_rect.topleft = (X+20, Y+90)
    text2_rect.topleft = (X+20, Y+140)
    text3_rect.topleft = (X+250, Y+140)

    text1_rect.height += 10      # Increase height by 20 pixels
    text2_rect.height += 10
    text3_rect.height += 10

    while not done:
        for event in pygame.event.get():      #check for events(key presses)
            # Draw buttons and rectangles
            #first button
            SCREEN.blit(text1_surface, text1_rect)
            pygame.draw.rect(SCREEN, green, text1_rect, 2)
            #second button
            SCREEN.blit(text2_surface, text2_rect)
            pygame.draw.rect(SCREEN, green, text2_rect, 2)
            #third button
            SCREEN.blit(text3_surface, text3_rect)
            pygame.draw.rect(SCREEN, green, text3_rect, 2)

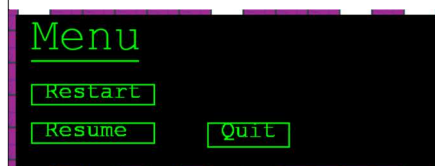
            #event handlers
            if event.type == pygame.QUIT:
                done = True
            if event.type == pygame.MOUSEBUTTONDOWN:
                if text1_rect.collidepoint(event.pos):
                    return "RESTART"
                if text2_rect.collidepoint(event.pos):
                    return "RESUME"
                if text3_rect.collidepoint(event.pos):
                    return "QUIT"

        pygame.display.update()
```

This is the function I made drawing on the learning from the reference above.

At the end of the program, it returns the status of which button was clicked.

This function can be called in paused so that nothing in the game is accessible at the same time.



After running this, I have realised that the quit (the cross in the top right corner) no longer works. To counteract this, I have added an “if event QUIT” so that I can break out of the while not done and have the game quit by using the cross in the top right corner, even if I’m not interacting with the Hero character in the game. The code is shown below.

```
if event.type == QUIT:      #check for quit
    done = True
    print(done)
    pygame.display.update()

pygame.quit()
```

Currently the way to bring up the menu is to click any random button whilst playing the game. I think I may leave it like this because there is a resume button

on the menu so it should be easy enough to resolve if the button is accidentally clicked.

First, to ensure that the menu would pop up when any other key was pressed except the already programmed ones, I used an “else” at the end of the keypress hero movements.

```
if Menu:
    Choice = MenuG()
    if Choice == "RESUME":
        Y = ((NumRows-1)*Thick + (NumRows-3)*Width)/2
        X = (Width + Thick)
        ExWidth = (Width*(NumCols))+(Thick*(NumCols-1))
        wid = ExWidth - (Width*2)
        #print(wid)
        rect = pygame.Rect(X, Y, wid, (Width*5))
        CellColour = colBG
        pygame.draw.rect(SCREEN, CellColour, rect, 0)
        Menu = False
        paused = False
        break #return to if not paused

    elif Choice == "RESTART":
        PlayGame(aMazeRecord, aCueFile, aUserRecord, HeroAvatarFile)
    else:
        GameOver = True
        paused = False
        break #return to game over
```

```
else:
    Menu = True
    yHero, xHero = posnHero[0]
    paused = True
```

The Menu if statement is within the paused while loop so the MenuG function should be called as soon as a key is pressed.

Resume – draws a rectangle to hide the menu as it is pushed to the back then sets the variables back to the defaults before breaking out of the while loop to stop the program from continuing to questions.

Restart – calls the PlayGame function again so that the game will initialise again and everything will be reset.

Quit – go to the GameOver Part of the game .

36.67. Questions

I also used references to help me understand and implement wrapping text and creating a text input box on screen within my questions GUI. These are the references that I drew on to help with my learning:

<https://stackoverflow.com/questions/49432109/how-to-wrap-text-in-pygame-using-pygame-font-font>³¹

<https://www.geeksforgeeks.org/how-to-create-a-text-input-box-with-pygame/>³²

Using my code from the previous menu GUI, I created the following subprogram:

```
def Questionpopup(Question):
    if int(Question[2]) == 1: #to allow the difficulty to be displayed to the User in the title
        Difficulty = "Easy"
    elif int(Question[2]) == 2:
        Difficulty = "Medium"
    elif int(Question[2]) == 3:
        Difficulty = "Hard"

    #ref: https://www.tutorialspoint.com/pygame/pygame_use_text_buttons.htm#:~:text=Let%20us%20try%20to%20displ
    done = False
    Y = ((NumRows-1)*Thick + (NumRows-3)*Width)/2 #Y positioning of the top left corner of the screen
    X = (Width + Thick) #X positioning of the top left corner of the screen
    ExWidth = (Width*(NumCols))+(Thick*(NumCols-1)) #total width of the game
    wid = ExWidth - (Width*2) #ensuring the width is the same on both sides
    # print(wid) #test print
    rect = pygame.Rect(X, Y, wid, (Width*5))
    CellColour = "black"
    pygame.draw.rect(SCREEN, CellColour, rect, 0) #GUI background

    #title of mini question gui.
    my_ft_font = pygame.freetype.SysFont('Courier New', 30) #title for the mini gui
    my_ft_font.underline = True
    #print(len>LastQs)) #test print for the question number
    my_ft_font.render_to(SCREEN, (X+20, Y+10), ("Q" + str(len>LastQs)) + " - "+Difficulty), (0, 255, 0)) #concat

    green = (0, 255, 0)
    # Button text font
    button_font = pygame.freetype.SysFont('Courier New', 30) #button settings from menu gui
    button_font.underline = False

    #ref for wrapping text after a certain number of characters
    #ref: https://stackoverflow.com/questions/49432109/how-to-wrap-text-in-pygame-using-pygame-font-font
    font = pygame.font.SysFont('Courier New', 20, bold=True) #load in the font type
    your_text = Question[0] #text is question to be answered
    txtX, txtY = X+20, Y+20 #location of text
    wraplen = 40 #number of characters before wrap
    my_wrap = textwrap.TextWrapper(width=wraplen) #use imported library textwrap
    wrap_list = my_wrap.wrap(text=your_text)
    # Draw one line at a time further down the screen
    for count in wrap_list:
        txtY = txtY + 35 #count the letters in the text
        Mtxt = font.render(f"{count}", True, green)
        SCREEN.blit(Mtxt, (txtX, txtY)) #add to screen

    # Render text buttons
    text1_surface, text1_rect = button_font.render(" SUBMIT ", green)

    # Set text into rectangles
    text1_rect.topleft = ((X+wid)- 175, Y+153)#edited

    text1_rect.height += 10 # Increase height by 20 pixels

    #ref for creating text input pygame
    #https://www.geeksforgeeks.org/how-to-create-a-text-input-box-with-pygame/
    base_font = pygame.font.SysFont('Courier New', 30)
    user_text = ''

    # create rectangle
    input_rect = pygame.Rect(X+20, Y+120, 300, 36)
    input_rect_color = "black" # Input box background color (blends in to gui background - not obvious)

    # Restrict input length (won't run off the page)
    max_length = 19 # Maximum characters allowed

    print(ExWidth/16)
    wraplen = ExWidth/16

    # Restrict input length (won't run off the page)
    max_length = ExWidth/26 # Maximum characters allowed
    print(ExWidth/63) #23 - long
    print(ExWidth/8) #52.25 - frogs
    print(ExWidth/16) #26.125 - frogs
```

Change from menu GUI to ensure it all fits on the page

The text will be wrapped so the question should be able to be seen.

After experimentation I have worked out the scale factor for the wrapped numbers so that it still works when the game has different dimensions.

After changing the wrap length to a scale factor, I decided to also do the same for the restricted input length to allow admins with longer answers to set longer horizontal games.

36.68. Event Handlers for this GUI

```
while not done:
    for event in pygame.event.get(): #check for events(key presses)
        if event.type == QUIT: #check for quit
            done = True
            #print(done) #test print to check quit functionality
        # Draw buttons and rectangles
        #button
        SCREEN.blit(text1_surface, text1_rect)
        pygame.draw.rect(SCREEN, green, text1_rect, 2)

        #event handlers
        if event.type == pygame.QUIT:
            done = True
        if event.type == pygame.KEYDOWN:

            # Check for backspace
            if event.key == pygame.K_BACKSPACE:

                # get text input from 0 to -1 i.e. end.
                user_text = user_text[:-1]

            elif event.key == pygame.K_RETURN:
                #print("User Input Submitted:", user_text) #test to check if input was correctly collected
                if user_text != "": #stops null being accidentally submitted
                    return user_text # Submit user input
            # Add new character if within max length
            # Unicode standard is used for string
            # formation
            elif len(user_text) < max_length:
                user_text += event.unicode

        # you can draw _input using pygame.font
        if event.type == pygame.MOUSEBUTTONDOWN:
            if text1_rect.collidepoint(event.pos):
                if user_text != "":
                    return user_text

        # Draw input rectangle
        pygame.draw.rect(SCREEN, input_rect_color, input_rect)

        # Render user text inside input rectangle
        text_surface = base_font.render(user_text, True, green)
        SCREEN.blit(text_surface, (input_rect.x + 5, input_rect.y + 5))

        # Adjust input rectangle width dynamically (but keep it within bounds)
        input_rect.w = max(200, text_surface.get_width() + 10)

        pygame.draw.rect(SCREEN, green, text1_rect, 2) #redraw button

    pygame.display.update()

pygame.quit() #after done
return
```

While not done is something I copied over from my menu GUI.

I also used the Quit part from the menu

I added this part for the text entry so that the answer would be returned if "submit" or the enter key are pressed.

I had to keep the game updated so that any previously deleted letters disappeared. And the Submit button always appeared on top.

36.69. How I linked this up to my temporary question system.

Previously, I created an interconnected web of subprograms to manage the questioning part of the maze. I used a subprogram called “questions” which output the questions in the console for answers. Now that this program exists, it can take the place of that temporary subprogram.

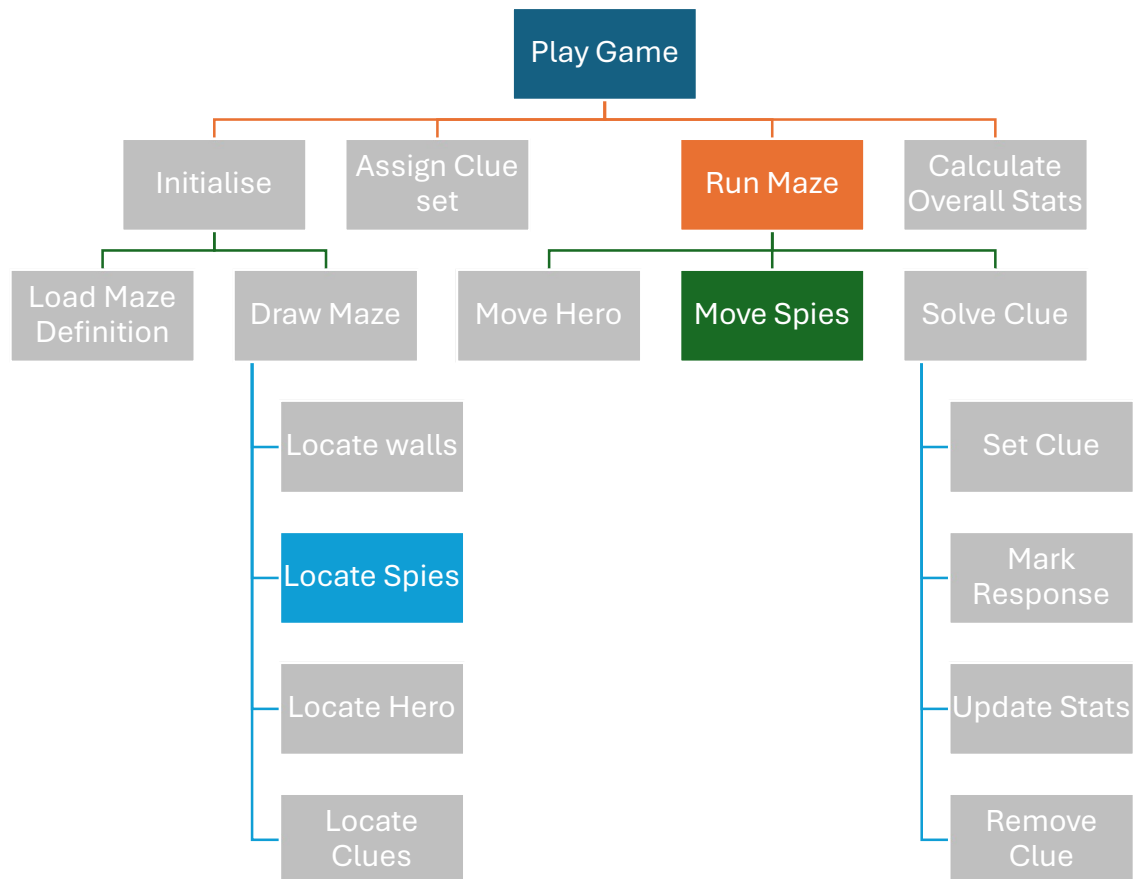
Here is how I linked it in: (in the paused part of PlayGame)

```
print(QuestionFull)
print(QuestionFull[0])
Answer = Questionpopup(QuestionFull)
AnsCheck = ClueAns(QuestionFull, Answer)
if AnsCheck:
```

36.70. To Do List at this point in the project:

- Move Spies
- Future Development Idea: I would like to be able to add the game window as part of the overall tkinter GUI framework, however this is quite a long way beyond my capability at the moment and I am not sure how compatibility would work for this.

36.71. I am now going to continue to consider the different elements of my decomposed system to move the Spy characters in my gameplay according to an algorithm that I will create. This area of focus is shown by the coloured boxes on the diagram below:



36.72. Managing the Spy movements

I have decided that I will have two different algorithms for spy movement.

1. The spy will go to a random next position in the maze
2. The spy will track the hero

I have decided that the split of the two algorithms for the spy movement will be 50% each and in the case of an odd number of spies there will be more tracking the hero.

This is an algorithm for generating the target for each spy. In half of the cases, the target will be the hero:

Random next position.

For each spy

Pick random direction(up down left right)

Get current spyPosn

Calculate next position according to direction

```

    If next position in not a wall then
        Move to new position
        //check for consequences
        If touching hero then
            move hero to random position
        end if
    endif //otherwise spy doesn't move this turn
next spy
//this happens each time the game loop iterates.

```

Algorithm to declare the go to positions.

```

Length = len(posnSpy)
length/2 = round(length/2) //if odd this number will be bigger than actual half
count = 0
for count in range(0 to length)           //loop as many times as the length of
posnSpy
    if count < (length/2) then             //selection to filter out half the results(this
half is bigger on odd numbers)
        position[count] = posnHero[0] //equal to initial position of hero
    else:                                  //other values(other half (maybe smaller))
        position[count] = random in posnEmpty //equal to any random
empty spot

```

36.73. Coding the spy position target

```
#spy movements
#get target
length = len(posnSpy)
length2 = round(length/2) #append original assignment to update when changed
for count in range(0,length):
    if count < (length2):
        if (randspyPosn[count] != posnHero[0])and (NewPosn not in posnWall):
            #change the spy position target to equal the hero position for half/more values
            randspyPosn[count] = posnHero[0]
            print(randspyPosn[count]) #test print
            print(posnHero[0]) #test print
        else:
            #if in a wall
            randspyPosn[count] = (currenty, currentx)
            print(randspyPosn[count])
            print(posnHero[0])
    else:
        if randspyPosn[count] == posnSpy[count]:
            #random position
            randspyPosn.append(random.choice(posnEmpty))
print(randspyPosn)
for spy in range(0, len(posnSpy)):
    pass
```

The code above provides the target for each spy but I haven't yet coded the spy movement. I will do this next and should replace "pass" with the code to move the spy.

36.74. Debugging the Spy Position Target Code

```
print(len(posnSpy)/2)
length = len(posnSpy)
length2 = round(length/2)
for count in range(0,length):
    if count < (length2):
        randspyPosn.append(posnHero[0])
    else:
        randspyPosn.append(random.choice(posnEmpty))
print(randspyPosn)
return #endprocedure
```

#if odd this number will be bigger than actual half
#loop as many times as the length of posnSpy
#selection to filter out half the results(this half is bigger)
#is initial position of hero
#other values(other half (maybe smaller))
#is any random empty spot

Using the code above as a basis, I edited this code to create a version of it in the Playgame function to update the target positions under certain condition existing.

I put this original assignment within the GetStartPosns procedure so it will only be done once.

```
#spy movements
length = len(posnSpy)
length2 = round(length/2)
for count in range(0,length):
    if count < (length2):
        if randspyPosn[count] != posnHero[0]:
            randspyPosn[count] = posnHero[0]
            print(randspyPosn[count])
            print(posnHero[0])
        else:
            if randspyPosn[count] == posnSpy[count]:
                randspyPosn.append(random.choice(posnEmpty))
```

#append original assignment to update when changed

LEFT
 BEFORE: row: 5 col: 13
 [(5, 12), (5, 12), (3, 8)]
 Hero hit wall
 [(1, 1), (1, 3), (1, 4), (1, 5),
 (3, 4), (3, 7), (3, 8), (3, 11),
 (5, 3), (5, 5), (5, 7), (5, 8),
 7, 5), (7, 7), (7, 8), (8, 1), (8, 11), (8, 12), (8, 13)]
 AFTER: row: 5 col: 13

Error: At the moment the position is not working correctly as it allows the randspypsn (the spies which change due to hero movement) to move inside the walls in error

To fix this error, I have added a secondary condition to the if statement.

```
if (randspyPosn[count] != posnHero[0]) and (NewPosn not in posnWall):
    randspyPosn[count] = posnHero[0]
    print(randspyPosn[count])
    print(posnHero[0])
else:
    randspyPosn[count] = (currenty, currentx)
```

I have used the current values which are also reassigned to posnHero[0] in the wall if statement later.

The target positions for the Spies of the two algorithms are below.

The position of the hero

It changes when the hero moves

The random generated position

It changes when the spy gets to this point.

```
[(5, 13), (5, 13), (3, 3)]
[(6, 13), (6, 13), (3, 3)]
[(5, 13), (5, 13), (3, 3)]
[(6, 13), (6, 13), (3, 3)]
```

36.75. Algorithm for moving the spy

After getting stuck on how to simplify the problem of moving the spy, I found this resource which told me how to break down the problem and from there, I was able to work out how to solve it:

https://www.reddit.com/r/gamemaker/comments/30ds45/help_pacman_game_ghost_path_code_help/33

This is my initial write up of an idea to solve the spy movement problem before I found the reference above. My initial idea would not have worked because it contains logic errors and mistakes, which I have removed in later iterations.

```
// find possible paths - remove non paths.
// up, down, right, left
directions = ["left", "right", "down", "up"]
if (StartY-1, StartX) in posnWall:
    directions.remove("up")
elif
if (StartY+1, StartX) in posnWall:
    directions.remove("down")
if (StartY, StartX+1) in posnWall:
    directions.remove("right")
if (StartY, StartX-1) in posnWall:
    directions.remove("left")
if prevposn != []
    if (StartY-1, StartX) in prevposn:
        directions.remove("up")
    if (StartY+1, StartX) in prevposn:
        directions.remove("down")
    if (StartY, StartX+1) in prevposn:
        directions.remove("right")
    if (StartY, StartX-1) in prevposn:
        directions.remove("left")
// compare to end position.
if (StartY > EndY):
    below = true
else:
    above = true
if (StartX > EndX):
    right = true
else:
    left = true
```

```
if direction = "up" and (directions[0] = "up")
    direction = "up"
elif above
    direction = random.choice(directions)
elif below and ("down" in directions):
    direction = "down"
elif right and ("right" in directions):
    direction = "right"
elif left and ("left" in directions):
    direction = "left"
else
    direction = random.choice(directions)
```

Below is an Algorithm I wrote (also before finding the reference above) building on the previous written algorithm and developing it to work slightly better (still with many mistakes and logic errors):

```
PrevPos = [] //in initialise
For spy in range 0 to len(posnSpy)
    //Calculate to get from point A->B
    StartY, StartX = posnSpy[spy]
    EndY, EndX = randspyPosn[spy]

    //find possible paths – remove non paths
    directions = ["up", "down", "right", "left"]

    //remove wall values
    if (StartY-1, StartX) in posnWall then
        directions.remove("down")
    if (StartY + 1, StartX) in posnWall then
        directions.remove("up")
    if (StartY, StartX+1) in posnWall then
        directions.remove("right")
    if (StartY, StartX-1) in posnWall then
        directions.remove("left")

    if prevPosn != [] then
        if (StartY-1, StartX) in prevPosn then
            directions.remove("up")
        if (StartY + 1, StartX) in prevPosn then
            directions.remove("down")
        if (StartY, StartX+1) in prevPosn then
            directions.remove("left")
```

```

        if (StartY, StartX-1) in prevPosn then
            directions.remove("right")

//compare to end position
if (StartY > EndY) then
    below = true
else:
    above = true

if (StartX > EndX) then
    right = true
else:
    left = true

//movement
if above and ( "up" in direction) then
    direction = "up"
else if below and ( "down" in direction) then
    direction = "down"
else if right and ( "right" in direction) then
    direction = "right"
else if left and ( "left" in direction) then
    direction = "left"
else
    direction = random.choice(directions)
end if

```

My final code for this subprogram is given on the next page. I use the reference highlighted above and include snips of the broken down problem from the reference. This solution does build on the two algorithms that I attempted above without the errors.

1. Look at the relative position of the ghost and target tile.

The black inserts are screenshots from the reference.

```
#compare to end position for spy in range(0, len(posnSpy)):
if (StartY < EndY):
    below = True
    above = False
    print("below")
else:
    above = True
    below = False
    print("above")

if (StartX < EndX):
    right = True
    left = False
    print("right")
else:
    left = True
    right = False
    print("left")
```

#Calculate to get from point A->B
StartY, StartX = posnSpy[spy] #position of spy
#print(posnSpy[spy])
EndY, EndX = randspyPosn[spy] #target position

Correct relationship comparison.
With test prints to check

To get the values that I need to compare.

2. Of the four possible directions, eliminate the one you're coming from and any that have walls. (This assumes your maze has no dead ends, like in Pac-Man, but if your maze has dead ends it's easy to check for.)

```
#find possible paths - remove non paths
directions = ["up", "down", "right", "left"]
nwdirections = ["up", "down", "right", "left"]

#remove wall values
if (StartY+1, StartX) in posnWall:
    directions.remove("down")
    nwdirections.remove("down")
if (StartY-1, StartX) in posnWall:
    directions.remove("up")
    nwdirections.remove("up")
if (StartY, StartX+1) in posnWall:
    directions.remove("right")
    nwdirections.remove("right")
if (StartY, StartX-1) in posnWall:
    directions.remove("left")
    nwdirections.remove("left")
```

First step create a list of directions.

- There are two so that one can be used in the case of a dead end

```
if PrevPosn != []:
    if (PrevPosn[spy] == ("down")) and ("up" in directions):
        directions.remove("up")
    if (PrevPosn[spy] == ("up")) and ("down" in directions):
        directions.remove("down")
    if (PrevPosn[spy] == ("right")) and ("left" in directions):
        directions.remove("left")
    if (PrevPosn[spy] == ("left")) and ("right" in directions):
        directions.remove("right")
```

These if statements remove any wall directions from the lists.
(They must be in separate ifs to ensure each one is executed)

Removes previous direction from the list of directions so that it can't turn around normally.

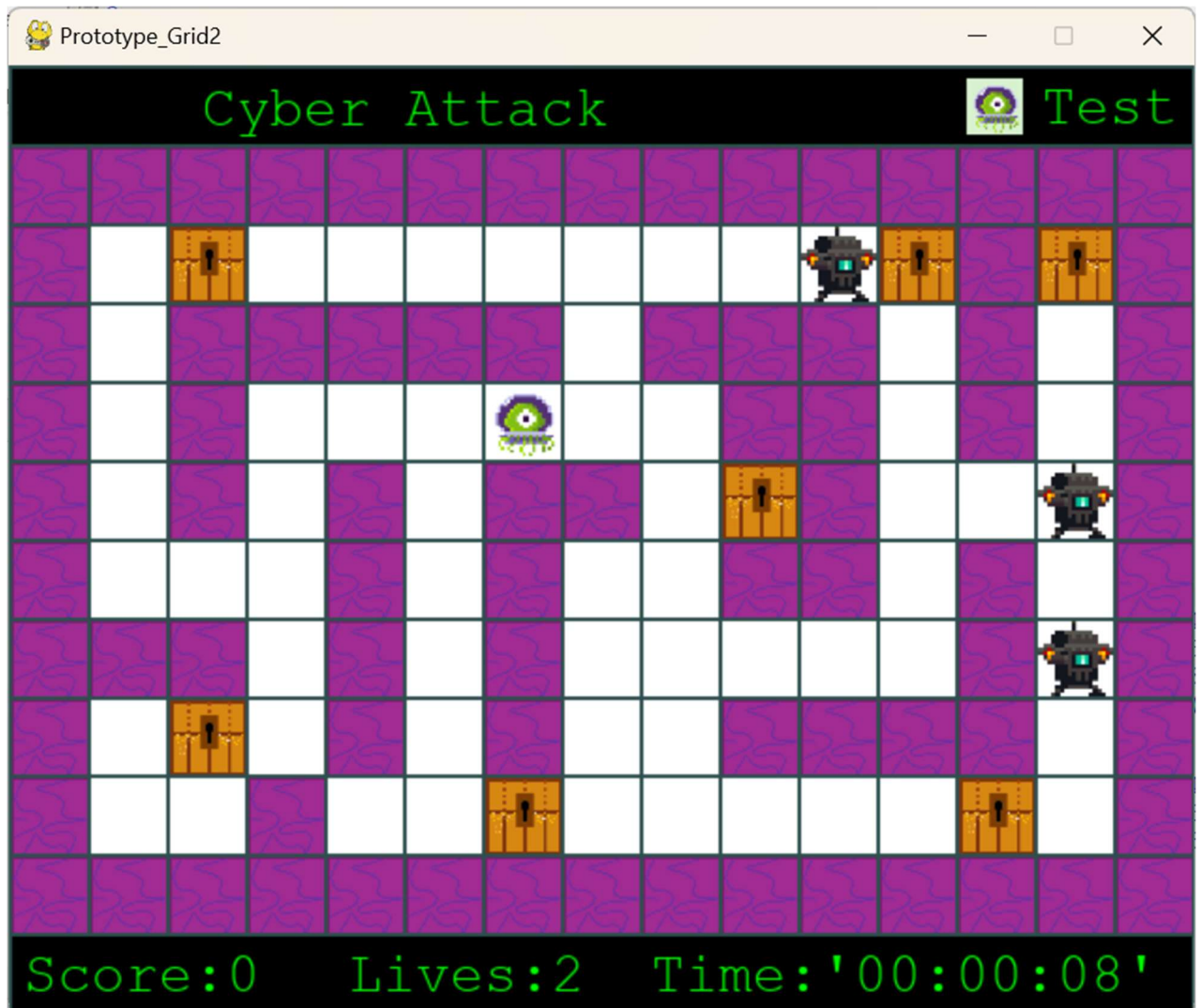
3. Check to see if any of the available directions move the ghost in the direction of the target tile. If so, pick one (doesn't matter which, really). 4. If none of the available directions are in the direction of the target tile, pick one (again, doesn't really matter).

```
#movement
if above and ("up" in directions):
    direction = "up"
elif below and ("down" in directions):
    direction = "down"
elif right and ("right" in directions):
    direction = "right"
elif left and ("left" in directions):
    direction = "left"
elif len(directions) < 1:
    direction = random.choice(nwdirections)
else:
    direction = random.choice(directions)
print(direction)
```

Selects a general direction where:

- if it can travel towards the target, it will do so unless none of the directions are available; and
- if this happens, then it will choose a random direction.

After implementing the code above, I found that the spies tended to follow each other around the map and get stuck together!



To try and resolve this I have coded a proximity change direction shown below:

```
#to re-shuffle the spies
#print(spy+1 < len(posnSpy))

if spy + 1 < len(posnSpy):
    Y, X = posnSpy[spy]
    Y1, X1 = posnSpy[spy + 1]
```

```
# Avoid proximity clumping
if abs(X1 - X) <= 2 and abs(Y1 - Y) <= 2:

    direction = random.choice(nwdirections)
```

This checks if the spy positions are close. If they are close, then the direction will either be randomised from the not wall options so that they go different ways eventually. (by setting the value to 2 the proximity won't be triggered through walls)

```

PrevPosn[spy] = direction
#print(directions, spy)      #test print

#determin new position
if direction == "up":
    newY = StartY - 1
    newX = StartX

elif direction == "down":
    newY = StartY + 1
    newX = StartX

elif direction == "left":
    newX = StartX - 1
    newY = StartY

elif direction == "right":
    newX = StartX + 1
    newY = StartY

#is new position in a wall
Posn = (newY, newX)

```

These directions are changed depending on which direction has been selected.

The code on the left is for the implementation of posnEmpty and also what to do if posnHero is hit by a spy

```

if Posn not in posnWall:
    posnSpy[spy] = Posn #update spy position
    if Posn in posnEmpty:
        posnEmpty.remove(Posn) #no longer empty
        posnEmpty.append((StartY, StartX)) #now empty

    if Posn in posnHero:
        print("Spy hit a Hero")
        print(posnHero)
        print()
        print(posnEmpty)
        #hit a spy
        currentposn = posnHero[0]
        Lives = Lives - 1

    #ref:https://stackoverflow.com/questions/306400/how-can-
    posnHero[0] = random.choice(posnEmpty) #teleports Hero
    posnEmpty.append(currentposn)

    #to remove the new position of the hero after teleportat
    yHero, xHero = posnHero[0]
    NewPosn = (yHero, xHero)
    posnEmpty.remove(NewPosn)
    print(posnEmpty)
    print(posnHero)

    if Lives == 0:
        GameOver = True

```

The not in posnWall is a double check that it is not possible for the spy to move into a wall.

To vary the speed of the spies based on the difficulty selected by the admin, I have created this if statement at the beginning to set the fixed frame count.

```

#different enemy speed of different levels
if aMazeRecord[2] == 1:
    diffcount = 30
elif aMazeRecord[2] == 2:
    diffcount = 25
else:
    diffcount = 20

```

36.76. My Updated Spy Movement Code

```

#spy movements
#get target
length = len(posnSpy)
length2 = round(length/2) #append original assignment to update when
#changed
for count in range(0,length):
    if count < (length2):
        if (randspyPosn[count] != posnHero[0])and (NewPosn not in posnWall):
            #change the spy position target to equal the hero position for
            #half/more values
            randspyPosn[count] = posnHero[0]
            ## print(randspyPosn[count]) #test print
            ## print(posnHero[0]) #test print
        else:
            #if in a wall
            randspyPosn[count] = (currenty, currentx)
            ## print(randspyPosn[count])
            ## print(posnHero[0])
    else:
        if randspyPosn[count] == posnSpy[count]:
            #random position
            randspyPosn[count] =(random.choice(posnEmpty))

```

```

#print(randspyPosn)

#now move spies
#frame counter
framecount = framecount+1
#print(framecount)      #test print
if framecount == diffcount:      #spy movement will change dependant on
    difficulty

    #Directions for randomised
    print(len(posnSpy))
    for spy in range(0 , len(posnSpy)):
        #Calculate to get from point A->B
        StartY, StartX = posnSpy[spy]      #position of spy
        #print(posnSpy[spy])
        EndY, EndX = randspyPosn[spy]      #target position

        #compare to end position
        if (StartY < EndY):      #comparison which if true the target is
below
            below = True
            above = False
            #print("below")      #test print
        else:
            above = True
            below = False
            #print("above")      #test print

        if (StartX < EndX):      #comparison which if true the target is
right
            right = True
            left = False
            #print("right")      #test print
        else:
            left = True
            right = False
            #print("left")      #test print

        #find possible paths - remove non paths
        directions = ["up", "down", "right", "left"]
        nwdirections = ["up", "down", "right", "left"]

        #remove wall values
        if (StartY+1, StartX) in posnWall:      #using coordinates
            directions.remove("down")
            nwdirections.remove("down")
        if (StartY - 1, StartX) in posnWall:
            directions.remove("up")
            nwdirections.remove("up")
        if (StartY, StartX+1) in posnWall:
            directions.remove("right")
            nwdirections.remove("right")
        if (StartY, StartX-1) in posnWall:
            directions.remove("left")
            nwdirections.remove("left")

```

```

#remove directions opposite to backwards
if PrevPosn != []:      #there won't be a preveous direction at the
start.
    if (PrevPosn[spy] == ("down")) and ( "up" in directions):
        directions.remove("up")

    if (PrevPosn[spy] == ("up")) and ("down" in directions):
        directions.remove("down")

    if (PrevPosn[spy] == ("right")) and ( "left" in directions):
        directions.remove("left")

    if (PrevPosn[spy] == ("left")) and ( "right" in directions):
        directions.remove("right")

#movement
#move in the direction if the target direction is available
if above and ("up" in directions):
    direction = "up"
elif below and ( "down" in directions):
    direction = "down"
elif right and ( "right" in directions):
    direction = "right"
elif left and ( "left" in directions):
    direction = "left"
elif len(directions) == 0:      #direction is empty - dead end
    direction = random.choice(nwdirections)
else:
    direction = random.choice(directions)

#to re-shuffle the spies
#print(spy+1 < len(posnSpy))

if spy + 1 < len(posnSpy):
    Y, X = posnSpy[spy]
    Y1, X1 = posnSpy[spy + 1]

    # Avoid proximity clumping
    if abs(X1 - X) <= 2 and abs(Y1 - Y) <= 2:      #abs to change
the value to positive
        direction = random.choice(nwdirections)      #random
available direction

PrevPosn[spy] = direction
#print(directions, spy)      #test print

#determin new position
if direction == "up":      #coordinates for up
    newY = StartY - 1
    newX = StartX

```

```

elif direction == "down":    #coordinates for down
    newY = StartY + 1
    newX = StartX

elif direction == "left":    #coordinates for left
    newX = StartX - 1
    newY = StartY

elif direction == "right":    #coordinates for right
    newX = StartX + 1
    newY = StartY

#is new position in a wall
Posn = (newY,newX)           #set coordinates

if Posn not in posnWall:      #triple check the possibility for a wall
coordinate

    posnSpy[spy] = Posn      #update spy position
    if Posn in posnEmpty:
        posnEmpty.remove(Posn)    #no longer empty
        posnEmpty.append((StartY, StartX))    #now empty

    if Posn in posnHero:
        print("Spy hit a Hero")    #test print
        print(posnHero)    #test print
        print()    #test print
        print(posnEmpty)    #test print
        ##
        ##
        ##
        #hit a spy
        currentposn = posnHero[0]    #set current position(may be
start and not have been set)
        Lives = Lives - 1    #increment lives

        posnHero[0] = random.choice(posnEmpty)    #teleports Hero to
New cell
        posnEmpty.append(currentposn)    #change empty

        #to remove the new position of the hero after teleportation
from posnEmpty
        yHero, xHero= posnHero[0]
        NewPosn = (yHero, xHero)
        posnEmpty.remove(NewPosn)
        ##
        ##
        print(posnEmpty)    #test print
        print(posnHero)    #test print

        if Lives == 0:    #check lives
            GameOver = True

framecount = 0

```

This is the code for the full spy movements as shown in PlayGame01.py